

AlphaEvolve: 用于科学与算法发现的编码智能体

亚历山大·诺维科夫^{*}, 银文^ũ, 马文·艾森伯格^{*}, 埃米利安·杜邦^{*}, 黄博森^{*}, 亚当·佐尔特·瓦格纳^{*}, 谢尔盖·希罗博科夫, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli and Matej Balog^{*}
Google DeepMind¹

本白皮书介绍了 AlphaEvolve, 一种进化式编码智能体, 它显著增强了最先进的大语言模型在极具挑战性任务上的能力, 例如解决开放科学问题或优化关键计算基础设施。AlphaEvolve 编排了一个自主的大语言模型流水线, 其任务是通过直接修改代码来改进算法。采用进化方法, 持续接收来自一个或多个评估器的反馈, AlphaEvolve 迭代地改进算法, 可能带来新的科学和实用发现。我们通过将其应用于多个重要计算问题来展示该方法的广泛适用性。当应用于优化谷歌大规模计算栈的关键组件时, AlphaEvolve 为数据中心开发了更高效的调度算法, 在硬件加速器的电路设计中找到了功能等价的简化方案, 并加速了支撑 AlphaEvolve 自身的大语言模型的训练。此外, AlphaEvolve 发现了一系列新颖的、可证明正确的算法, 在数学和计算机科学的一系列问题上超越了最先进的解决方案, 显著扩展了先前自动发现方法的范围 (Romera-Paredes 等人, 2023)。值得注意的是, AlphaEvolve 开发了一种搜索算法, 找到了使用 48 次标量乘法计算两个 4×4 复值矩阵相乘的方法; 这是 56 年来首次在该场景下对 Strassen 算法的改进。我们相信 AlphaEvolve 及类似的编码智能体能够在科学和计算的许多领域对改进问题解决方案产生重大影响。

1. 引言

发现新的高价值知识, 例如做出新颖的科学发现或开发具有商业价值的算法, 通常需要一个漫长的过程, 包括构思、探索、对无望假设的回溯、实验和验证。近年来, 人们对使用大语言模型 (Large Language Models) 来自动化这一过程的重要部分产生了浓厚的兴趣。成功的希望源于近期大语言模型令人惊叹的能力 [32, 76], 它们可以通过测试时计算来增强自身能力, 以及结合语言生成和行动的智能体的兴起 [88, 114]。这些进展在一系列既定基准上提升了性能, 并加速了假设生成 [34] 和实验设计 [7, 43] 等以发现为导向的任务。然而, 让大语言模型流水线完全实现全新的科学或实用发现仍然具有挑战性。

在本白皮书中, 我们提出了一个名为 AlphaEvolve 的大语言模型代码超优化智能体, 它利用进化计算和基于大语言模型的代码生成来应对这一挑战。AlphaEvolve 专注于广泛的科学与工程

¹ 参见致谢和作者信息部分。^{*}Equal 贡献。

发现问题的候选发现可以被自动评估。它将候选者（例如，新的数学对象或实用的启发式方法）表示为算法，并使用一组大语言模型来生成、批判和进化这些算法的池。大语言模型引导的进化过程通过代码执行和自动评估来落地。这种评估机制使 AlphaEvolve 能够避免基础大语言模型 [44] 的任何错误建议。

AlphaEvolve 中的进化过程利用了现代大语言模型（LLMs）对反馈的响应能力，从而能够发现与初始候选池在语法和功能上显著不同的候选方案。它既适用于以发现新算法为内在目标的问题，也适用于更广泛的一类问题——在这些问题中，所关注的解本身并非算法，但算法可以描述如何构造或找到该解。在后一种情况下，发现算法只是工具性目标，但事实证明，与直接搜索解相比，这是一种出人意料的有效策略 [83]。

将进化方法与编码大语言模型相结合的想法此前已在多种专门场景中探索过。特别是，AlphaEvolve 是对 FunSearch [83] 的实质性增强（见表 1），后者利用大语言模型引导的进化来发现启发式方法，以构造新颖的数学对象或驱动在线算法的运行。此外，相关方法也已用于模拟机器人策略发现 [57]、符号回归 [35, 89] 以及组合优化启发式函数综合 [63] 等任务。与这些系统相比，AlphaEvolve 利用最先进的大语言模型来进化大型代码片段，这些代码实现了跨越多个函数和组件的复杂算法。因此，它在规模和通用性上显著超越了前代系统。

<i>FunSearch</i> [83]	<i>AlphaEvolve</i>
evolves single function	evolves entire code file
evolves up to 10-20 lines of code	evolves up to hundreds of lines of code
evolves code in Python	evolves any language
needs fast evaluation ($\leq 20\text{min}$ on 1 CPU)	can evaluate for hours, in parallel, on accelerators
millions of LLM samples used	thousands of LLM samples suffice
small LLMs used; no benefit from larger	benefits from SOTA LLMs
minimal context (only previous solutions)	rich context and feedback in prompts
optimizes single metric	can simultaneously optimize multiple metrics

表 1 | AlphaEvolve 与我们此前智能体的能力及典型行为。

虽然自动化评估度量的使用为 AlphaEvolve 带来了关键优势，但它也是一种局限——尤其是，它将需要人工实验的任务排除在我们的范围之外。由于数学、计算机科学和系统优化中的问题通常允许自动化评估度量，我们在 AlphaEvolve 上的工作集中于这些领域。具体而言，我们利用 AlphaEvolve 在算法设计和构造性数学中的若干著名开放问题上取得进展，并优化谷歌大规模计算栈中的关键层。

在算法设计领域，我们关注发现快速矩阵乘法算法这一基本问题，此前已有更专门的人工智能方法应用于该问题 [26]。尽管 AlphaEvolve 是通用型方法，但其表现超越了 [26]，在 14 种矩阵乘法算法上刷新了最先进水平；尤为突出的是，对于 4×4 矩阵，AlphaEvolve 通过发现一种仅需 48 次乘法即可完成 4×4 复值矩阵相乘的算法，改进了 Strassen (1969) 的算法。在数学领域，我们考虑了一系列广泛的开放问题，这些问题可通过发现性质优于所有已知构造（对象）的新构造来取得进展，其优劣依据给定的数学定义判定。我们将 AlphaEvolve 应用于超过 50 个此类问题，在 $\sim 75\%$ 的问题上达到了已知最优构造的水平（其中许多构造很可能已是最优）。在 $\sim 20\%$ 的问题上，AlphaEvolve 超越了最先进水平，发现了新的、可证明更优的构造。这包括对 Erdős 提出的最小重叠问题 [25] 的改进，以及在 11 维接吻数问题 [8, 31] 上构造的改进。

最后，我们将 AlphaEvolve 应用于四个工程问题，这些问题横跨谷歌计算栈的不同层级：为谷歌集群管理系统发现调度启发式算法、优化用于训练大语言模型的矩阵乘法核函数、优化 TPU 内部使用的算术电路，以及优化变换器中注意力机制的运行时间。由于这些组件需要长期反复运行，任何改进都具有极高价值。

2. AlphaEvolve

AlphaEvolve 是一个编码智能体，它编排一个包含大语言模型查询在内的自主计算流水线，并生成解决用户指定任务的算法。从高层来看，其编排过程是一个进化算法，逐步开发出在与任务相关的自动化评估指标上得分更高的程序。AlphaEvolve 的高层概览如图 1 所示，图 2 给出了展开视图。

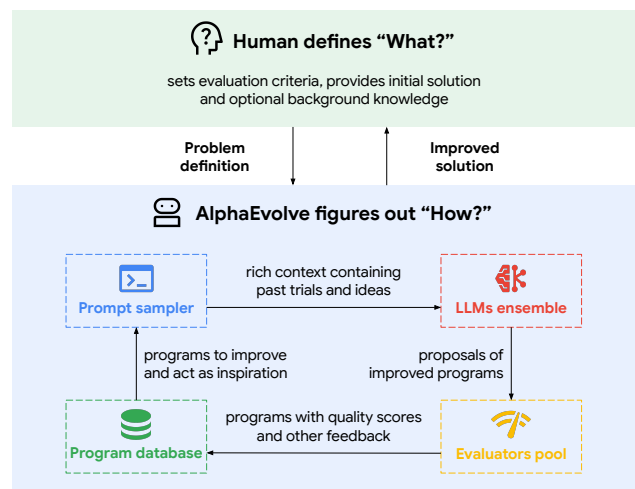


图 1 | AlphaEvolve 高层概览。

2.1. 任务规范

评估。由于 AlphaEvolve 处理的是具有机器可评分解决方案的问题，用户必须提供一种自动评估生成方案的机制。该机制以函数 h 的形式实现，将解决方案映射为一组标量评估指标。按照惯例，这些指标以最大化为目标。在当前设置中， h 通常实现为 2 这些发现的算法以及我们其他新的数学结果可在 https://colab.research.google.com/github/google-deepmind/alphaevolve_results/blob/master/mathematical_results.ipynb 找到。

找到。

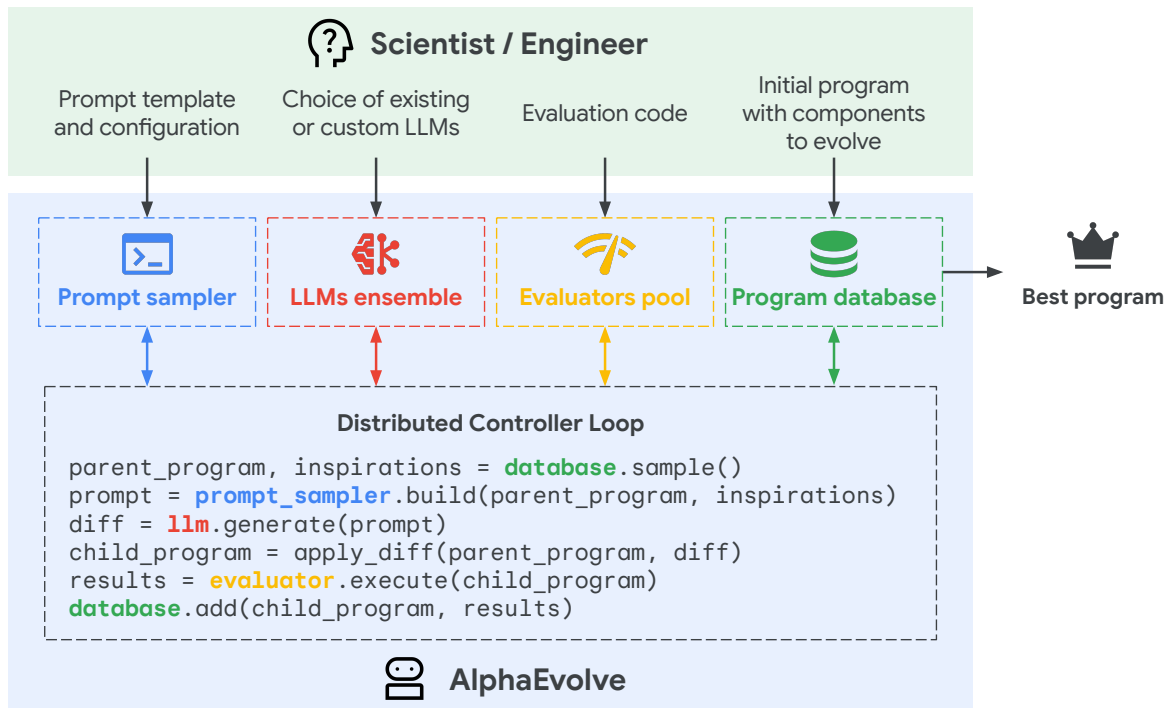


图 2 | AlphaEvolve 发现过程的扩展视图。用户提供初始程序（标记出待演化的组件）、评估代码和可选配置（第 2.1 节）。随后 AlphaEvolve 启动演化循环。提示采样器利用程序数据库中的程序构建丰富的提示（第 2.2 节）。基于这些提示，大语言模型生成代码修改（差异），并应用这些修改创建新程序（第 2.3 节）。然后由评估器对这些程序进行评分（第 2.4 节），有前景的解决方案被注册回程序数据库（第 2.5 节），从而推动迭代发现越来越好的程序。

作为一个 Python 函数，名为 `evaluate`，具有固定的输入/输出签名，返回一个标量字典。

根据应用场景的不同，执行该函数可能只需在单台设备上运行数秒，也可能需要启动大规模计算。对于数学问题，函数 h 通常非常简单。例如，当希望找到满足给定性质的最大可能图时， h 调用演化代码生成一个图，检查该性质是否成立，然后简单地返回图的大小作为得分。在更复杂的情况下，函数 h 可能涉及执行一个演化搜索算法，或训练和评估一个机器学习模型。

应用程序编程接口。为了支持在代码库中演化多个组件，AlphaEvolve 公开了一个输入应用程序编程接口，可以将代码块标注为待系统演化；参见图 3a 的说明。这种设计便于将其集成到现有代码库中，只需在代码中添加特殊标记（`# EVOLVE-BLOCK-START` 和 `# EVOLVE-BLOCK-END`）作为注释，即可实现最小化改动。

此类演化代码块中的任何用户提供的代码，都作为 AlphaEvolve 待改进的初始解决方案，其余代码则构成一个骨架，将演化出的各个部分连接起来，以便它们能够从 `evaluate` 中被调用。虽然这个初始实现必须是完整的，但它可以是初步的——例如，由返回适当类型常量的单行函数组成。

选择抽象方式的灵活性。 AlphaEvolve 可以以截然不同的方式应用于同一问题——尤其是当演化出的程序不是最终输出，而是发现解决方案的手段时。例如，AlphaEvolve 可以以原始字符串表示形式演化解决方案（如经典演化算法中那样）；演化一个具有确定形式的函数，该函数指定如何从头构建解决方案（[83] 中采用的方法）；演化一个定制搜索算法，在某个固定计算预算内寻找解决方案；甚至可以将中间解决方案与搜索算法共同演化，使得每个搜索算法都专门针对进一步改进某个特定的中间解决方案而定制。

我们发现，不同的问题适合不同的抽象层级。例如，我们假设对于具有高度对称解的问题，演化构造函数是有利的，因为这些函数往往更简洁 [83]；而对于具有非对称解的问题，演化定制搜索算法效果更好。

2.2. 提示采样

由于 AlphaEvolve 利用了最先进的大语言模型（SOTA LLMs），它支持多种类型的定制，并提供长上下文作为主演化提示的一部分。该提示包含从程序数据库中采样的多个先前发现的解决方案，以及关于如何对特定解决方案提出修改的系统指令。除了这些关键要素之外，用户还可以通过不同方式进一步根据自身特定需求定制提示，例如以下几种。

- 显式上下文：关于所解决问题的详细信息，例如固定的人工编写指令、方程、代码片段或相关文献（如 PDF 文件）。
- 随机格式化：带有由人工提供的替代项的模板占位符，以增加多样性，并使用单独配置文件中提供的概率分布进行实例化。
- 渲染的评估结果：通常包括一个程序、该程序的执行结果以及由评估函数分配的分数。
- 元提示演化：由大语言模型自身在额外的提示生成步骤中提出的指令和上下文，在与解决方案程序类似的独立数据库中共同演化。

2.3. 创造性生成

为了驱动演化过程，AlphaEvolve 利用了最先进的大语言模型（LLM）的能力，其主要作用是消化先前开发的解决方案的信息，并提出新颖且多样的改进方案。尽管 AlphaEvolve 与模型无关，但在消融实验中我们观察到，随着底层大语言模型的改进，AlphaEvolve 的性能会越来越好（见第 4 节）。

```
# EVOLVE-BLOCK START
"""Image classification experiment in jaxline."""

import jax
...
# EVOLVE-BLOCK-END

...

# EVOLVE-BLOCK-START
class ConvNet(hk.Module):
    def __init__(self, num_classes): ...
    def __call__(self, inputs, is_training): ...

def sweep():
    return hyper.zipit([...])
# EVOLVE-BLOCK-END

...

def evaluate(eval_inputs) -> dict[str, float]:
    ...
    return metrics
```

(a) 

The current model uses a simple ResNet architecture with only three ResNet blocks. We can improve its performance by increasing the model capacity and adding regularization. This will allow the model to learn more complex features and generalize better to unseen data. We also add weight decay to the optimizer to further regularize the model and prevent overfitting. AdamW is generally a better choice than Adam, especially with weight decay.

```
<<<<<< SEARCH
    self._block1 = ResNetBlock(num_channels)
    self._block2 = ResNetBlock(num_channels * 2, stride=2)
    self._block3 = ResNetBlock(num_channels * 4, stride=2)
=====
    self._block1 = ResNetBlock(num_channels)
    self._block2 = ResNetBlock(num_channels, stride=1)
    self._block3 = ResNetBlock(num_channels * 2, stride=2)
    self._block4 = ResNetBlock(num_channels * 2, stride=1)
    self._block5 = ResNetBlock(num_channels * 4, stride=2)
    self._block6 = ResNetBlock(num_channels * 4, stride=1)
>>>>>> REPLACE
<<<<<< SEARCH
    def optimizer(self, learning_rate):
        return optax.adam(learning_rate)
=====
    def optimizer(self, learning_rate):
        return optax.adamw(learning_rate, weight_decay=1e-4)
>>>>>> REPLACE
```

(c) 

Act as an expert software developer. Your task is to iteratively improve the provided codebase. [...]

- Prior programs

Previously we found that the following programs performed well on the task at hand:

top_1_acc: 0.796; neg_eval_log_loss: 0.230; average_score: 0.513

```
"""Image classification experiment in jaxline."""
[...]
```

```
class ConvNet(hk.Module):
    """Network."""

    def __init__(self, num_channels=32, num_output_classes=10):
        super().__init__()
        self._conv1 = hk.Conv2D(num_channels, kernel_shape=3)
        self._conv2 = hk.Conv2D(num_channels * 2, kernel_shape=3)
        self._conv3 = hk.Conv2D(num_channels * 4, kernel_shape=3)
        self._logits_module = hk.Linear(num_output_classes)
    [...]
```

- Current program

Here is the current program we are trying to improve (you will need to propose a modification to it below).

top_1_acc: 0.862; neg_eval_log_loss: 0.387; average_score: 0.624

```
"""Image classification experiment in jaxline."""
[...]
```

```
class ConvNet(hk.Module):
    """Network."""

    def __init__(self, num_channels=32, num_output_classes=10):
        super().__init__()
        self._conv1 = hk.Conv2D(num_channels, kernel_shape=3)
        self._block1 = ResNetBlock(num_channels)
        self._block2 = ResNetBlock(num_channels * 2, stride=2)
        self._block3 = ResNetBlock(num_channels * 4, stride=2)
        self._logits_module = hk.Linear(num_output_classes)
    [...]
```

SEARCH/REPLACE block rules:
[...]

Make sure that the changes you propose are consistent with each other. For example, if you refer to a new config variable somewhere, you should also propose a change to add that variable.

Example:
[...]

Task
Suggest a new idea to improve the code that is inspired by your expert knowledge of optimization and machine learning.

Describe each change with a SEARCH/REPLACE block.

(b) 

图 3 | AlphaEvolve 应用于演化监督学习流程的示例。所有代码片段均经过缩写，省略号（...）表示跳过的行。(a) 用户提供的文件，其中标记了用于演化的代码块，以及可调用为当前代码版本评分的特殊评估函数。(b) 提供给大语言模型的组装提示示例。(c) 大语言模型生成的输出示例。(c) 中提出的差异将应用于提示 (b) 中显示的“当前程序”，然后将修改后的程序发送给评估器。评估器将调用 (a) 中的评估函数，以获得新提出程序的分。

输出格式。当 AlphaEvolve 要求大语言模型修改现有代码时，尤其是在较大的代码库中，它会要求以特定格式的差异块序列提供更改：

```
<<<<<<< SEARCH
  # Original code block to be found and replaced
=====
  # New code block to replace the original
>>>>>>> REPLACE
```

此处，<<<<<<< SEARCH 和 ===== 之间的代码是当前程序版本中要匹配的确切片段。===== 和 >>>>>>> REPLACE 之间的代码是将替换原始片段的新片段。这允许对代码的特定部分进行有针对性的更新。

当被演化的代码非常短，或者完整重写比小修改更合适时，AlphaEvolve 可以配置为指示大语言模型直接输出整个代码块，而不是使用差异格式。

使用的模型。AlphaEvolve 采用大语言模型集成。具体来说，我们利用 Gemini 2.0 Flash 和 Gemini 2.0 Pro 的组合。这种集成方法使我们能够平衡计算吞吐量与生成解决方案的质量。Gemini 2.0 Flash 具有较低的延迟，能够实现更高的候选生成速率，从而增加每单位时间探索的想法数量。同时，Gemini 2.0 Pro 具有更强的能力，能够偶尔提供更高质量的建议，这些建议可以显著推进演化搜索，并有可能带来突破。这种策略性混合通过最大化评估想法的数量，同时保留由更强大模型驱动的实质性改进潜力，优化了整体发现过程。

2.4. 评估

为了跟踪 AlphaEvolve 的进展并选择在未来世代中传播哪些想法，大语言模型提出的每个新解决方案都会被自动评估。原则上，这个过程相当于简单地对生成的解决方案执行用户提供的评估函数 h 。在实践中，AlphaEvolve 支持可选的机制，使这种评估更加灵活和高效：

- 评估级联（假设检验）：用户可以指定难度递增的测试用例集合，使得新解决方案只有在所有早期阶段都取得足够有前景的结果时，才会在下一阶段进行评估。这有助于更快地剔除前景较差的解决方案。此外，新解决方案最初在小规模上进行评估，然后再接受主要测试用例的检验，以便尽早过滤掉有缺陷的程序。
- 大语言模型生成的反馈：在某些应用中，理想的解决方案具有某些难以在用户提供的评估函数中精确捕捉的特征

h ；例如，所发现程序的简洁性。这些属性可以使用单独的大语言模型调用进行评分，并添加到分数字典中以引导演化，或者在未满足标准时用于丢弃解决方案。

- 并行化评估：AlphaEvolve 的样本效率使得花费约 100 计算小时来评估任何新解成为可行。然而，除非对单个评估进行并行化以减少其墙钟时间，否则这会减慢新一代出现的速度，限制进化算法连续应用多次突变的能力。在许多应用中，评估是极易并行的（例如，从多个随机初始化运行搜索算法），这使得 AlphaEvolve 能够通过对评估集群的异步调用来分配这项工作。

多重评分。AlphaEvolve 允许优化多个用户提供的评分，即在一种或多种评估度量下进化出获得高分的对象。这既有内在价值，也有工具价值。虽然在多个应用中我们确实关心为多种评估度量开发解决方案（或一个在所有度量上同时表现强劲的解决方案），但我们发现，即使某一度量特别受关注，针对多个度量进行优化也常常能改善单一目标度量的结果。或许这是因为在不同评估标准下表现出色的程序往往具有不同的结构或逻辑，通过将这些多样化、高性能程序的示例（每个都代表“好”的不同定义）纳入提供给语言模型的提示中，我们可以刺激生成更多多样化的候选解决方案，从而增加发现对目标度量高度有效的新方法的可能性。

2.5. 进化

在进化过程中，AlphaEvolve 不断生成越来越多的解决方案，并附带评估结果（评分和程序输出）。这些解决方案存储在一个进化数据库中，其主要目标是在未来世代中最佳地重新呈现先前探索过的想法。设计此类数据库的一个关键挑战是平衡探索与利用，以持续改进最佳程序，同时保持多样性以鼓励对整个搜索空间的探索。在 AlphaEvolve 中，进化数据库实现了一种算法，该算法受 MAP elites 算法 [74] 和基于岛屿的种群模型 [83, 97] 的组合启发。

2.6. 分布式流水线

AlphaEvolve 被实现为一个异步计算流水线（使用 asyncio Python 库），其中许多计算并发运行，每个计算在其下一步依赖于另一个尚未完成的计算的结果时会阻塞（等待）。更具体地说，异步流水线包括一个控制器、LLM 采样器和评估节点。整个流水线针对吞吐量（而不是任何单个计算的速度）进行了优化，以便在特定的总体计算预算内最大化可以提出和评估的想法数量。

$\langle m, n, p \rangle$	best known [reference]	AlphaEvolve
$\langle 2, 4, 5 \rangle$	33 [42]	32
$\langle 2, 4, 7 \rangle$	46 [93]	45
$\langle 2, 4, 8 \rangle$	52 [93]	51
$\langle 2, 5, 6 \rangle$	48 [93]	47
$\langle 3, 3, 3 \rangle$	23 [52]	23
$\langle 3, 4, 6 \rangle$	56 [48]	54
$\langle 3, 4, 7 \rangle$	66 [91]	63
$\langle 3, 4, 8 \rangle$	75 [91]	74
$\langle 3, 5, 6 \rangle$	70 [48]	68
$\langle 3, 5, 7 \rangle$	82 [91]	80
$\langle 4, 4, 4 \rangle$	49 [95]	48
$\langle 4, 4, 5 \rangle$	62 [47]	61
$\langle 4, 4, 7 \rangle$	87 [93]	85
$\langle 4, 4, 8 \rangle$	98 [95]	96
$\langle 4, 5, 6 \rangle$	93 [48]	90
$\langle 5, 5, 5 \rangle$	93 [72]	93

表 2 | 表示一个 $m \times n$ 矩阵与一个 $n \times p$ 矩阵乘积的张量 $\langle m, n, p \rangle$ 的秩的上界，即计算该矩阵乘积所需的标量乘法次数。除了此处展示的例子外，对于所有参数 $m, n, p < 5$ ，AlphaEvolve 要么匹配要么超越了已知的最佳解，并提供了精确算法（完整结果见附录中的表 3）。对于 $\langle 3, 4, 7 \rangle$ 、 $\langle 4, 4, 4 \rangle$ 和 $\langle 4, 4, 8 \rangle$ ，AlphaEvolve 发现的算法使用复值乘法，可用于复数或实数矩阵的精确乘法。本表中展示的分解可以在随附的 Google Colab 中找到。

3. 结果

3.1. 通过寻找张量分解的新颖算法实现更快的矩阵乘法 从加速机器学习计算到实现逼真的计算机图形学，矩阵乘法是支撑计算机科学中众多关键算法和应用的基本运算。自 Strassen 的开创性工作 [95] 以来，人们已经知道，将两个矩阵相乘的丰富算法空间可以表示为给定三维张量到秩一张量的分解。分解的秩（项数）精确指定了计算矩阵乘积所需的标量乘法次数。因此，为了开发更快的矩阵乘法算法，需要找到特定张量的低秩分解。这个问题已经通过多种方法解决，从专门的交替最小二乘求解器 [93] 到深度强化学习 [26] 和自定义搜索算法 [47]；然而，尽管经过数十年的努力，即使对于两个 3×3 矩阵相乘的简单情况，最小可达到的秩仍然未知，这展示了该问题的难度。

从问题描述和标准的基于梯度的算法（包括初始化器、重构损失函数和 Adam 优化器 [50]）出发，AlphaEvolve 能够

开发出超越现有方法的复杂张量分解算法。为评估每个演化出的程序，我们选择一组矩阵乘法目标，并使用第 2.4 节描述的评估级联以多个随机种子初始化运行该算法。性能随后以每个目标上达到的最佳（最低）秩以及达到该秩的种子比例来衡量，为 AlphaEvolve 提供爬山信号。为确保分解的精确性并避免任何潜在的数值误差，在评估时，我们将每个元素四舍五入到最接近的整数或最接近的半整数；并且，为鼓励算法生成接近整数的解，我们在 LLM 的提示中以自然语言包含了这一要求。

在表 2 中，可以看到 AlphaEvolve 开发的各种算法在 14 个不同的矩阵乘法目标上改进了最新技术水平。值得注意的是，对于两个 4×4 矩阵的乘法，递归应用 Strassen 的算法 [95] 可得到秩（标量乘法次数）为 49 的算法，该算法适用于任意域。对于在含有 2 个元素的域中乘法的非常特殊的情况，Fawzi 等人 [26] 找到了一个秩为 47 的算法。56 年来，在任意特征为 0 的域上设计秩小于 49 的算法一直是一个未解决的问题。³ AlphaEvolve 是第一个找到秩为 48 的算法来乘以两个 4×4 复值矩阵的方法。

如图 4 所示，AlphaEvolve 对初始程序进行了重大修改，引入了若干新颖的想法来设计越来越好的算法。虽然表 2 中的大多数结果（包括 $\langle 4, 4, 4 \rangle$ ）都是从简单的初始程序获得的，但我们发现对于某些参数，用我们自己的想法（例如向评估函数添加随机性或使用进化方法）来初始化程序可以进一步提升性能，这凸显了研究人员与 AlphaEvolve 之间科学协作的可能性。

3.2. 为广泛的开放数学问题寻找定制搜索算法 数学研究的一个重要前沿涉及根据某种测度发现具有最优或接近最优性质的物体或构造。例子包括寻找几何形状的密集堆积 [29] 到识别满足特定组合或分析约束的函数或集合（例如，[39, 40, 70, 104]）。进展通常依赖于找到一个超越所有先前已知例子的单一构造，从而为最优值建立新的下界或上界。我们证明 AlphaEvolve 是探索这些问题中固有广阔搜索空间的强大工具，成功应对了各种开放的数学挑战。

为了评估其能力，我们将 AlphaEvolve 应用于一组精心挑选的 50 多个数学问题，涵盖数学的五个以上不同分支，包括分析、组合学、数论和几何，并在众多特定参数设置（例如，不同的维度或大小）下进行评估。在 75% 的情况下，AlphaEvolve 重新发现了已知的最佳构造，在 20% 的情况下，它发现了一个比先前已知最佳构造更好的新对象，从而改进了最先进水平。在所有这些情况下，初始起点都是简单或随机的构造。这些结果强调了 AlphaEvolve 作为数学研究多功能工具的广泛潜力。

³ 存在使用少于 49 次乘法的算法，但它们不对应于矩阵乘法张量的分解，并且不能递归地应用于乘以更大的矩阵。

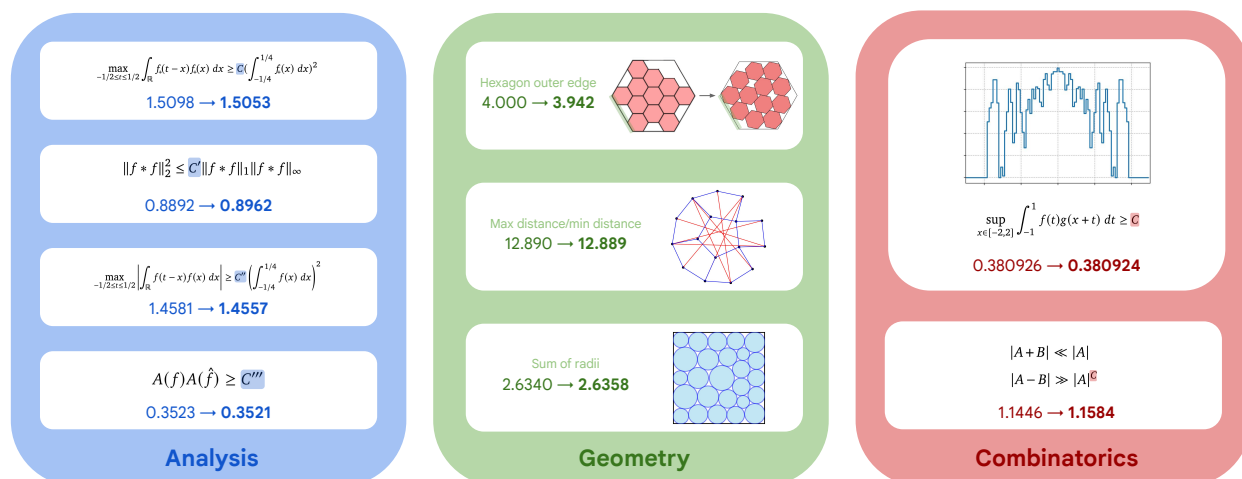


图 5 | 使用 AlphaEvolve 发现的最先进数学构造示例。 AlphaEvolve 的多功能性使我们能够解决分析（自相关和不确定性不等式）、几何（填充和最小/最大距离问题）以及组合数学（Erdős 最小重叠问题以及有限集的和与差）中的问题。

此处所用 AlphaEvolve 配置的一个显著优势在于其多功能性与应用速度。其核心方法论聚焦于演化启发式搜索程序（详见下文），能够快速部署于多种数学构造问题与猜想，相较于传统定制方法，通常所需的前期问题特定专家调整更少。尽管深厚的数学洞见自然有助于问题表述与搜索空间定义，AlphaEvolve 往往展现出自主发现有效搜索模式与攻击策略的能力，通过识别问题景观中的微妙结构。这使得能够对众多不同问题进行高效、大规模探索。

促成这些发现的关键方法论创新在于 AlphaEvolve 能够演化启发式搜索算法，而非直接演化构造本身。对于许多问题，尤其是那些目标函数评估迅速的问题——这在数学中很常见——我们采用了迭代精化策略。每一代 AlphaEvolve 的任务是演化一个代表搜索启发式的程序。该程序被赋予固定的时间预算（例如 1000 秒），并展示由上一最佳启发式找到的最佳构造。其目标是利用这一起点和分配的时间来寻找更优的构造。因此，演化过程选择那些在改进已高质量解方面有效的启发式。最终构造往往是 AlphaEvolve 发现的一系列不同、专门化启发式的结果——早期启发式擅长从随机或简单初始状态获得大幅增益，而后期启发式则擅长对近最优配置进行微调。这种多阶段、自适应搜索策略的自动发现难以手动复制，并被证明对于超越最先进水平至关重要。

以下是对 AlphaEvolve 产生新结果的部分问题的高层描述。完整问题列表与细节见附录 B。

- **分析**

- **自相关不等式。**AlphaEvolve 能够改进若干自相关不等式的最佳已知界。
- **不确定性原理。**AlphaEvolve 通过优化不确定性原理构造 [33]，为傅里叶分析中的一个问题生成了精细配置，从而得到了略优的上界。

- **组合数学与数论**

- **埃尔德什最小重叠问题。**AlphaEvolve 为最小重叠问题 [25] 建立了新的上界，略微改进了此前的记录 [40]。

- **几何与填充**

- **接吻数问题。**在 11 维中，AlphaEvolve 改进了接吻数的下界，找到了一种 593 个互不重叠的单位球同时与一个中心单位球相切的配置，超过了此前 592 的记录 [31]。
- **填充问题。**AlphaEvolve 在装箱问题中取得了多项新成果，例如在形状内放置 N 个点以最小化最大与最小距离之比、以最高效方式将各种多边形装入其他多边形，以及海布隆问题中避免小面积三角形的点集变体 [29]。

完整问题列表见附录 B，AlphaEvolve 发现的新构造可在随附的 Google Colab 中找到。关于这些问题及所用方法的更多示例和细节将在后续论文中提供。这些发现大多来自外部数学家 Javier Gomez Serrano 和 Terence Tao 向我们建议的开放问题，他们还就如何将其最佳表述为 AlphaEvolve 的输入提供了建议。这凸显了 AlphaEvolve 这类人工智能驱动的发现引擎与人类数学专长之间协同合作的潜力。

3.3. 优化谷歌计算生态系统 除了前面章节介绍的科学应用外，这里我们展示 AlphaEvolve 如何用于提升关键任务基础设施的性能并产生实际影响。

3.3.1. 改进数据中心调度 将计算作业高效调度到机器集群上是一个关键的优化问题，尤其是在由 Borg [102] 编排的谷歌数据中心规模下。该任务涉及根据作业资源需求和机器容量将作业分配到可用机器。低效的分配可能导致资源搁浅：当机器因某种资源（如内存）耗尽而无法再接受作业，但其他资源（如 CPU）仍有空闲时。调度效率的提升可以回收这些搁浅资源，从而在相同资源量下完成更多作业

```

1 定义 alpha_evolve_score(required, 空闲)：中央处理
2 器剩余量 = 所需中央处理器 / 空闲中央处理器
3 内存剩余量 = 所需内存 / 空闲内存
4
5 返回 -1.0 * (中央处理器残差 + 内存残差 + 内存残
6 差 / 中央处理器残差 + 中央处理器残差 /
7 mem_residual)

```

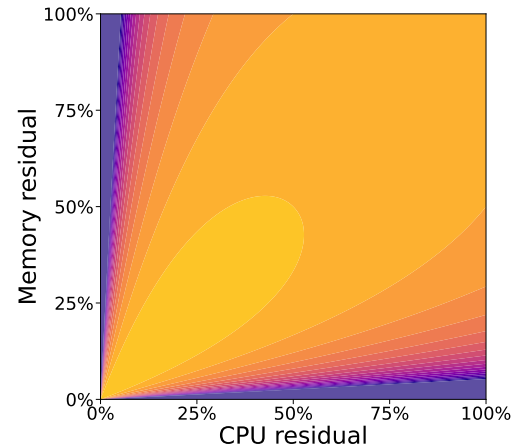


图 6 | 左：AlphaEvolve 发现的启发式函数，针对谷歌的工作负载和容量定制。右：启发式评分函数的可视化。黄色区域表示高分，紫色区域表示低分。

计算足迹的恢复。这种恢复对于在不按比例增加资源消耗的情况下满足日益增长的计算需求至关重要。此外，该问题具有挑战性，因为它将典型的工程难题（如可调试性和规模）与经典困难的装箱问题相结合。

我们通过将在线作业调度问题构建为具有两个变量的向量装箱问题来应对这一挑战。在此背景下，机器代表具有明确中央处理器和内存容量的箱子，而传入的作业则是具有特定资源需求的项目。启发式函数将待处理作业的中央处理器和内存需求以及潜在机器的中央处理器和内存可用性作为输入，然后输出该机器的优先级分数。随后，博格调度器将待处理作业分配给由启发式函数确定的优先级分数最高的机器，同时兼顾其他目标。由于该启发式函数仅影响博格已确定为可用且能够运行每个待处理作业的机器的排名，因此生成的调度决策在构造上实际上是正确的。

早期版本的阿尔法进化被用于发现一个极其简单却有效的启发式函数（如图 6 所示），该函数从生产环境中的现有函数演化而来。我们使用数据中心的模拟器，基于谷歌机群中工作负载和容量的历史快照，为阿尔法进化提供反馈。我们在近期工作负载和容量的未见测试数据集上衡量阿尔法进化启发式函数的性能，以确保泛化能力。观察到阿尔法进化的启发式函数优于生产环境中的函数后，我们将阿尔法进化的启发式函数推广到整个机群。部署后对谷歌机群的测量证实了模拟器结果，表明该启发式函数平均持续恢复谷歌机群 0.7% 的计算资源，否则这些资源将被搁置。选择阿尔法进化而非深度强化学习方法，是因为其代码解决方案不仅性能更优，而且在可解释性、可调试性、可预测性和部署便捷性方面具有明显优势——这些都是关键任务系统所必需的品质。

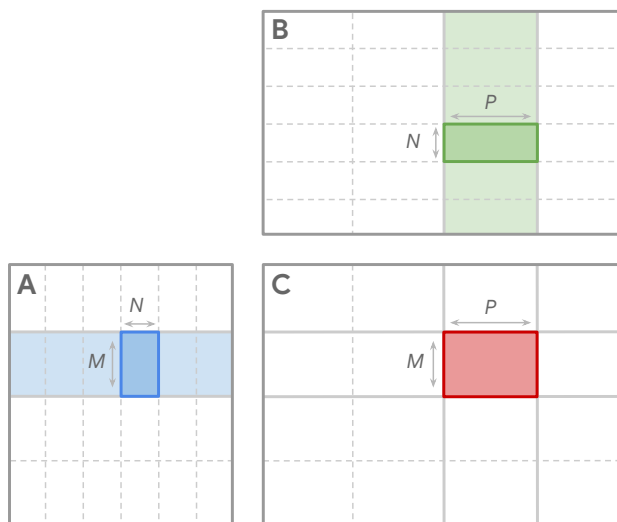


图 7 | 矩阵乘积的平铺启发式问题可视化 $AB = C$ 。创建一个能自动为所有输入形状选择正确平铺尺寸 (M, N, P) 的启发式方法是困难的，因为必须了解矩阵乘法单元的最优形状和内存容量、周围操作的内存需求、融合到核函数中的额外操作以及底层编译器的复杂性等细节。

3.3.2. 增强双子座核函数工程

训练像双子座这样的大型模型需要大量的计算资源。双子座构建于 JAX[9] 之上，而 Pallas 是 JAX 的一个扩展，它支持编写定制的、高度专门化的程序（核函数），这些程序针对硬件加速器上的最优执行进行了定制。因此，高效的 Pallas 核函数对于优化双子座的训练性能至关重要。核函数优化的一个关键方面是调整矩阵乘法运算的平铺策略（见图 7）。该技术涉及将大型矩阵乘法计算划分为更小的子问题，以更好地平衡计算与数据移动，这是加速整体计算的关键。传统上，核函数工程师依赖基于搜索的自动调优或手工设计的启发式方法来确定各种输入形状的近似最优平铺配置。基于搜索的调优会中断研究工作流程，每次输入形状变化都需要重新调优。相反，手工设计有效的平铺启发式方法由于其复杂性而成为主要的工程瓶颈，需要深入理解核函数功能和硬件复杂性。高性能启发式方法的关键优势在于其能够在任意输入形状下提供高性能。因此，为了加快为新兴硬件设计高性能核函数的速度，并简化模型开发人员对其的使用，我们旨在促进启发式方法的生成过程。

我们通过采用阿尔法进化（AlphaEvolve）来优化一个用于训练双子座（Gemini）的重要矩阵乘法核函数的平铺启发式，以应对这一挑战。目标是使该核函数的实际运行时间最小化。阿尔法进化通过提出候选代码，迭代地探索和优化该核函数的平铺启发式，旨在真实张量处理单元（TPU）加速器上针对各种输入形状最小化运行时间。由于阿尔法进化优化的是该核函数的平铺策略而非改变其底层数学运算，因此核函数的正确性在构造上得以保持。

为了构建阿尔法进化的训练和评估数据集，我们自动从核函数用户处收集真实的核函数输入形状。其中一半输入形状构成训练集，为进化过程提供优化目标。其余输入形状构成评估集，用于测试所得启发式的通用适用性。

这种自动化方法使阿尔法进化能够发现一种启发式，该启发式在所有核函数上相比现有专家设计的启发式平均实现了 23% 的核函数加速，并相应地将双子星的整体训练时间减少了 1%。此外，阿尔法进化的使用显著缩短了核函数优化时间，从数月的专门工程努力缩短到仅数天的自动化实验。这种加速加快了优化核函数的部署，使核函数工程师能够将专业知识投入到更具战略性的高层优化问题上。此外，阿尔法进化提供了一条自动化手动调优过程并改善双子星核函数使用体验的途径。阿尔法进化发现的平铺启发式已部署于生产环境，直接提升了双子星的训练效率以及双子星团队的研究和工程速度。这一部署也标志着一个新颖的实例，即双子星通过阿尔法进化的能力优化了自身的训练过程。

3.3.3. 辅助硬件电路设计 专用硬件，如谷歌的张量处理单元（TPU），对于实现大规模运行现代人工智能系统所需的资源效率至关重要。然而，设计新的计算机芯片是一个复杂且耗时的过程，通常需要数年时间。寄存器传输级（RTL）优化是该过程中的关键步骤，涉及手动重写硬件描述以改进功耗、性能和面积等指标，需要高技能工程师进行数月的迭代。

在这项工作中，AlphaEvolve 被要求优化矩阵乘法单元内关键 TPU 算术电路的一个已经高度优化的 Verilog 实现。优化目标是在保持组件核心功能的同时，减少面积和功耗。至关重要的是，最终方案必须通过稳健的验证方法，以确认修改后的电路保持功能正确性。AlphaEvolve 能够找到一个简单的代码重写，移除了不必要的位，这一更改已由 TPU 设计师验证其正确性。虽然这一具体改进也被下游综合工具独立发现，但 AlphaEvolve 在 RTL 阶段的贡献展示了其精炼源 RTL 并在设计流程早期提供优化的能力。

这一改进被集成到即将推出的 TPU 中，代表了 Gemini 通过 AlphaEvolve 对 TPU 算术电路的首次直接贡献，为未来的贡献铺平了道路。AlphaEvolve 的一个关键优势是它直接用 Verilog（硬件工程师使用的标准语言）传达建议的更改，从而建立信任并简化采用。这项早期探索展示了一种新颖的方法，即由大语言模型驱动的代码进化辅助硬件设计，有可能缩短产品上市时间。

3.3.4. 直接优化编译器生成代码 变换器 (Transformer) 架构 [100] 被用于大多数现代神经网络, 从大型语言模型到阿尔法折叠 (AlphaFold) [1]。变换器的核心计算是注意力机制 [4], 最常使用闪速注意力 (FlashAttention) [22] 实现。在我们的技术栈中, 闪速注意力作为加速器核函数在帕拉斯 (Pallas) 中实现, 由 JAX 中处理输入准备和输出后处理的高层代码包装。机器学习编译器 (XLA[77]) 随后将该实现转换为一系列中间表示 (IR), 每个中间表示都为在特定硬件上执行添加更多细节。在这些阶段, 改进内存访问编排或计算调度的决策可以显著减少特定硬件上的运行时间。

我们挑战阿尔法进化 (AlphaEvolve) 直接优化封装闪速注意力核函数以及前后处理代码的 XLA 生成的中间表示。我们优化了一个对应于在图形处理器上大规模推理的高影响力变换器模型的配置, 目标是最小化模块的整体执行时间。这是一项特别具有挑战性的任务, 因为 (1) 中间表示是为调试目的而设计的, 而非供开发者直接编辑, 并且 (2) 它是编译器生成的且已经高度优化。阿尔法进化提出的每次修改都在随机输入上与参考 (未修改) 代码进行核对, 以确保整个优化过程中的数值正确性。代码的最终版本由人类专家严格确认对所有可能输入均正确。

阿尔法进化能够为中间表示暴露的两个抽象层次提供有意义的优化。首先, 针对目标配置的闪速注意力核函数加速了 32%。其次, 阿尔法进化在核函数输入和输出的前后处理中发现了改进, 使该部分加速了 15%。这些结果证明了阿尔法进化优化编译器生成代码的能力, 为将发现的优化纳入现有编译器以用于特定用例提供了潜力, 或者从长远来看, 将阿尔法进化纳入编译器工作流程本身。

4. 消融实验

我们在两个任务上进行了消融实验: 寻找更快的矩阵乘法的张量分解 (第 3.1 节) 和计算接吻数的下界 (第 3.2 节), 旨在理解 AlphaEvolve 以下组件的有效性。

- **进化方法。** AlphaEvolve 采用进化方法, 将先前生成的程序存储在数据库中, 并在后续迭代中用于获得更好的程序。为了分析进化的重要性, 我们考虑了一种替代方法, 该方法反复将相同的初始程序输入语言模型。我们将这种方法称为“无进化”。
- **提示中的上下文。** AlphaEvolve 使用具有大上下文窗口的强大语言模型, 通过在提示中提供特定于问题的上下文, 可以显著改善其输出。为了测试上下文的重要性, 我们考虑了一种替代方法, 即在提示中不添加显式上下文。我们将这种方法称为“提示中无上下文”。

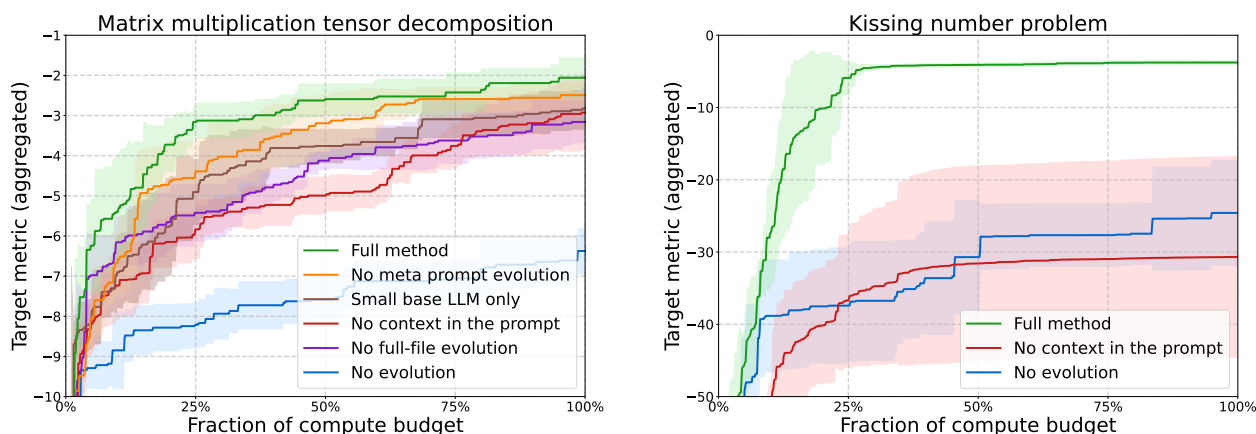


图 8 | 左：AlphaEvolve 在寻找低秩张量分解以实现更快矩阵乘法问题上的消融。右：AlphaEvolve 在寻找球堆积以改进接吻数问题上的消融。 每条曲线显示了随着计算预算增加，单个设置的平均性能（目标度量值越高越好），该平均值基于所有考虑的目标。阴影表示目标内标准差，基于 AlphaEvolve 使用不同随机种子初始化的三次独立运行的平均值。

- **元提示。** AlphaEvolve 还使用元提示来改进提供给语言模型的提示。这使其有可能超越人类提示者所能获得的性能。为了测试元提示的有效性，我们在张量分解任务中禁用了它。我们将这种方法称为“无元提示进化”。
- **全文件演化。** 与以往如趣味搜索（FunSearch）等方法不同，阿尔法演化（AlphaEvolve）能够演化整个代码库，而非仅聚焦于单一函数。为检验全文件演化的重要性，我们在张量分解的背景下考虑一种仅演化损失函数的替代方案，并将该方法称为“无全文件演化”。
- **强大的语言模型。** 阿尔法演化（AlphaEvolve）依赖小型与大型语言模型的混合，以获得高度多样化的样本。为理解该组件的重要性，我们考虑一种仅使用单个小型基础模型的替代方案，并将该方法称为“仅小型基础大语言模型”。

图 8 展示了包含全部组件的阿尔法演化（AlphaEvolve）方法以及上述各种替代方案的结果。可以看出，每个组件都对结果的显著提升起到了作用。

5. 相关工作

演化方法。 阿尔法演化（AlphaEvolve）延续了演化或遗传编程 [54] 的悠久研究传统，即反复使用一组变异和交叉算子来演化程序池 [5, 51]。特别是，经典演化技术在符号回归应用 [66, 87]、自动化科学 [21] 或算法 [16] 发现以及调度 [118] 问题中取得了成功。然而，这些方法面临的一个挑战是

方法之一是使用手写的演化算子，这些算子可能难以设计，并且可能无法捕捉领域的重要特性。相比之下，AlphaEvolve 使用大语言模型来自动构建这些算子——它利用大语言模型的世界知识来变异程序，而无需预先定义一组允许的变异操作。

AlphaEvolve 之前有一系列近期工作将大语言模型与演化相结合；具体来说，它扩展了 Romera-Paredes 等人提出的 FunSearch 系统 [83]，该系统是一种数学发现方法。FunSearch 随后被用于下游任务，例如为贝叶斯优化学习采集函数 [2]、发现认知模型 [13]、计算图之间的距离 [103] 或组合竞争性编程 [101]。AlphaEvolve 在三个关键方面超越了 FunSearch 及其近期的重新实现 [24]。首先，FunSearch 只允许演化单个 Python 函数，而 AlphaEvolve 允许对整个代码库进行演化，这些代码库可以用多种编程语言编写。其次，FunSearch 优化单个目标函数，而 AlphaEvolve 提供了执行多目标优化的能力。第三，FunSearch 中的大语言模型相对较小，并且仅在代码上训练。相比之下，AlphaEvolve 使用前沿大语言模型以及丰富的自然语言上下文和反馈形式。正如本文所展示的，这些扩展使 AlphaEvolve 能够解决 FunSearch 无法处理的重要挑战性问题。

这一类别中的其他工作包括 Lehman 等人 [57] 的方法，该方法使用大语言模型引导的演化过程来为一组模拟机器人发现程序化策略；以及 Hemberg 等人 [41] 用于代码综合的方法。类似的方法已在多个科学和数学任务中得到应用，包括符号回归 [35, 89]、发现组合优化的启发式方法 [63, 115, 117] 以及合成分子结构 [105]。大语言模型引导的演化也被用于通过增强大语言模型提示 [27] 和搜索神经架构 [14, 73] 来改进人工智能系统。AlphaEvolve 与这些方法的不同之处在于其规模、灵活性以及对广泛领域的普遍适用性。

近期一些研究通过引入互补思想，增强了基于大语言模型引导的进化基本范式。例如，苏里纳等人 [96] 通过强化学习持续微调大语言模型，以补充进化过程。格雷耶利等人 [35] 利用大语言模型引导的概念学习步骤增强进化过程，该步骤将程序池中表现优异的程序总结为自然语言。要理解这些思想在 AlphaEvolve 所运行的规模上带来的益处，还需要进一步研究。

进化方法在近期的人工智能联合科学家 (AI Co-Scientist) 研究 [34] 中也得到了应用，该研究旨在利用不同智能体分别完成假设发现、假设排序和文献综述等任务，从而实现科学发现的自动化。人工智能联合科学家以自然语言表示科学假设及其评估标准，而 AlphaEvolve 专注于进化代码，并利用程序化评估函数指导进化过程。这一选择使我们能够大幅规避大语言模型的幻觉问题，从而使 AlphaEvolve 能够持续进行大量时间步的进化。尽管如此，原则上仍可将两种方法结合，形成一种能够灵活融合自然语言与程序化表达的方法。

超优化与算法发现。AlphaEvolve 可视为一种代码超优化方法，因为它利用执行反馈迭代改进初始程序。代码超优化的思想可追溯至 20 世纪 80 年代 [69]；在大语言模型出现之前，解决该问题的方法包括系统枚举 [69]、遗传搜索 [20]、蒙特卡洛采样 [86] 以及深度强化学习 [68]。此外，在专注于单一问题（如矩阵乘法）的有限场景中，已有 AlphaTensor 等系统能够发现可证明正确的算法 [26]。

最近，涌现出一批基于大语言模型（LLM）的超优化与算法发现方法。这些文献建立在 LLM 在编码任务上的成功之上，其最佳例证或许是在（模拟）编程竞赛中的表现，如 AlphaCode[60]。例如，LLM 智能体已被用于优化 GPU 内核中的某些运算，如注意力运算 [15] 或更通用的用户指定运算 [56]。此外，还有研究利用 LLM 发现新颖的进化算法 [55]、训练语言模型 [58] 以及优化仓库级计算机 [61]。其他近期工作 [108] 也提出使用多个相互对话的 LLM 智能体来完成数学与编码任务。

尽管先前利用 LLM 进行算法发现的工作已展现出有前景的结果，AlphaEvolve 将其用于进化算法的方法使我们能够应对显著更具挑战性的问题，如第 3 节所示。

面向科学与数学发现的人工智能。过去十年间，人工智能系统已被应用于广泛的科学学科与任务，从蛋白质结构预测 [46] 到量子物理 [6, 84] 再到气候科学 [53]。特别是，近期涌现出大量基于 LLM 的方法，针对多个学科中的科学问题，如材料科学 [45, 71, 94, 119]、化学 [12, 64]、生物信息学 [67, 85]、地球科学 [79] 和量子物理 [30, 78]（相关综述见 [36, 65, 81]）。

其中许多方法利用 LLM 自动化科学发现过程中的若干不同阶段 [37, 59, 106, 109, 112]，例如生成与排序假设和想法 [38, 90]。在这些方法中，与 AlphaEvolve 尤为相关的是使用 LLM 引导的树搜索算法 [11] 或 LLM 引导的进化算法 [34, 113, 120] 的方法。其他工作则利用 LLM 优化实验规划与设计 [7, 10, 43, 75] 或实验执行与工作流 [28, 62, 82, 105, 116]。最后，还有工作聚焦于数据分析阶段 [80]。AlphaEvolve 与大多数此类方法的区别在于其使用程序化假设表示与评估度量。

人工智能系统也为纯数学的进展做出了贡献 [23]。在此背景下，FunSearch 方法 [24, 83] 确立了 LLM 引导的进化作为发现数学命题见证与反例的强大工具——该问题与寻找数学命题的形式化与非形式化证明互补 [3, 19, 98, 99, 110, 111]。

6. 讨论

AlphaEvolve 展示了在进化框架内将最先进的大语言模型与自动化评估度量相结合所产生的惊人力量，这既能带来对数十年前数学问题的新发现，也能为高度优化的计算栈带来实用的改进。

有趣的是，AlphaEvolve 通常允许以不同方式处理同一问题：直接搜索解决方案、寻找一个从头构建它的函数，或进化出一个搜索算法来找到它。以不同方式应用 AlphaEvolve 会带来不同的偏差（例如，寻找构造性函数可能有利于发现高度对称的对象 [83]），因此适用于不同的问题。

AlphaEvolve 也可以被视为一种测试时计算智能体，通过其进化过程，显著增强了基础大语言模型的能力（与例如重复采样相比）。一方面，这可以看作是一个有力的证明，展示了机器反馈如何能够维持测试时计算规模的扩展，直至达到能够做出新科学发现和极具价值的实用优化的程度。另一方面，一个自然的下一步将是考虑将 AlphaEvolve 增强的基础大语言模型性能蒸馏到下一代基础模型中。这不仅具有内在价值，而且很可能提升下一版 AlphaEvolve 的能力。

除了蒸馏之外，AlphaEvolve 能够做出提高其自身基础设施及其（未来版本的）基础大语言模型效率的实用发现，这一点也引人入胜。目前，这些收益是适度的，并且改进下一版 AlphaEvolve 的反馈循环周期约为数月。然而，随着这些改进，我们设想，建立更多具有稳健评估函数的环境（问题）的价值将得到更广泛的认可，这反过来将带来更多高价值的实用发现。

AlphaEvolve 的主要局限性在于它处理的是那些能够设计出自动化评估器的问题。虽然数学和计算科学中的许多问题确实如此，但在自然科学等领域，只有部分实验可以被模拟或自动化。尽管 AlphaEvolve 确实允许通过大语言模型（LLM）对想法进行评估，但这并非我们优化的场景。然而，同期工作表明这是可行的 [34]，一个自然的步骤是将这两种场景连接起来，由大语言模型在过渡到实现阶段之前对高层想法提供反馈，而在实现阶段则可通过代码执行获得机器反馈。

致谢

我们感谢 Michael Figurnov 审阅本白皮书；感谢 Alhussein Fawzi、Bernardino Romera-Paredes 和 Ankit Anand 的早期探索与深刻讨论；感谢 Stig Petersen 和 Demis Hassabis 的支持与建议；感谢 JD Velasquez 在管理实际应用方面提供的宝贵建议；以及所有 AlphaEvolve 的早期用户和合作者，他们多样化的用例和深刻的反馈使 AlphaEvolve 成为适用于广泛应用的更强大、更通用的工具。我们衷心感谢以下个人对本白皮书中所强调应用的宝贵贡献：

Terence Tao、Javier Gomez Serrano 和 Jordan Ellenberg 提出了具体的开放数学问题，并就如何为 AlphaEvolve 最佳地表述这些问题提供了建议；Bogdan Georgiev、Ray Jiang 和 Johannes Bausch 为将 AlphaEvolve 应用于此类问题做出了贡献。

Mohammadamin Barekatin、Patrick Heisel、Chase Hensel、Robert O’Callahan 和 Pengming Wang 共同领导了数据中心调度应用；Federico Piccinini、Sultan Kenjeyev 和 Andrea Michi 做出了重要贡献；Kieran Milan、Daniel Mankowitz、Cosmin Paduraru、Calin Cascaval、Tammo Spalink 和 Natasha Antropova 提供了有益的建议；Aaron Gentleman、Gaurav Dhiman、Parthasarathy Ranganatha 和 Amin Vahdat 审阅了本工作。

Yanislav Donchev 领导了 Gemini 核函数工程应用；Richard Tanburn 做出了重要贡献；Justin Chiu 和 Julian Walker 提供了有益的建议；Jean-Baptiste Alayrac、Dmitry Lepikhin、Sebastian Borgeaud、Koray Kavukcuoglu 和 Jeff Dean 审阅了本工作。

蒂穆尔·西特迪科夫 (Timur Sitdikov) 领导了将应用扩展到张量处理单元 (TPU) 电路设计的工作；乔治·罗蒂瓦尔 (Georges Rotival) 提供了电路评估基础设施；柯克·桑德斯 (Kirk Sanders)、斯里坎特·德瓦拉卡纳特 (Srikanth Dwarakanath)、因德拉尼尔·查克拉博蒂 (Indranil Chakraborty)、克里斯托弗·克拉克 (Christopher Clark) 验证并确认了张量处理单元设计中的结果；维诺德·奈尔 (Vinod Nair)、塞尔吉奥·瓜达拉马 (Sergio Guadarrama)、迪米特里奥斯·维蒂尼奥蒂斯 (Dimitrios Vytiniotis) 和丹尼尔·贝洛夫 (Daniel Belov) 提供了有益的建议；克里·竹中 (Kerry Takenaka)、杰夫·迪恩 (Jeff Dean)、斯里达尔·拉克什马纳穆尔蒂 (Sridhar Lakshmanamurthy)、帕塔萨拉蒂·兰加纳坦 (Parthasarathy Ranganathan) 和阿明·瓦赫达特 (Amin Vahdat) 审阅了本工作。

本杰明·切蒂乌伊 (Benjamin Chetoui)、谢尔盖·列别杰夫 (Sergei Lebedev)、亚历山大·别利亚耶夫 (Alexander Belyaev)、亨宁·贝克尔 (Henning Becker)、奥列格·希什科夫 (Oleg Shyshkov) 和阿利娅·哈萨诺娃 (Aliia Khasanova) 在加速线性代数 (XLA) 修改方面提供了帮助并给出了有益建议；乔治·阿雷纳 (Giorgio Arena)、马尔科·科尔内罗 (Marco Cornero) 和塞巴斯蒂安·博登斯坦 (Sebastian Bodenstein) 审阅了本工作。

作者信息

这些作者贡献相同：亚历山大·诺维科夫 (Alexander Novikov)、阮武 (Ngân V`u)、马文·艾森伯格 (Marvin Eisenberger)、埃米利安·杜邦 (Emilien Dupont)、黄宝森 (Po-Sen Huang)、亚当·若尔特·瓦格纳 (Adam Zsolt Wagner)、谢尔盖·希罗博科夫 (Sergey Shirobokov)、鲍里斯拉夫·科兹洛夫斯基 (Borislav Kozlovskii) 和马泰·巴洛格 (Matej Balog)。

贡献。 A.N. 和 M.B. 设计并实现了 AlphaEvolve 的初始版本。M.B.、A.N.、N.V. 和 P.K. 制定了项目愿景并界定了问题范围。N.V. 和 P.-S.H. 监督了实际应用。E.D. 和 M.E. 在 F.J.R.R. 和 M.B. 的参与下，实现了用于迭代 AlphaEvolve 的第一个基准问题。A.N. 和 M.E. 在 S.S.、P.-S.H. 的贡献以及 M.B.、E.D.、A.Z.W. 和 N.V. 的参与下，开发了 AlphaEvolve 的最终版本。A.N.、S.S.、P.-S.H. 和 M.E. 维护了 AlphaEvolve 的底层基础设施。M.E. 和 E.D. 在 F.J.R.R. 的参与下，使用 AlphaEvolve 发现了矩阵乘法的新算法。A.Z.W. 在 A.M.、M.E. 和 A.N. 的帮助下，致力于将 AlphaEvolve 应用于开放数学问题。A.N. 为 Borg 调度应用做出了贡献。P.-S.H. 和 N.V. 致力于 Gemini 核函数工程应用。P.-S.H. 和 A.N. 为张量处理单元电路设计应用做出了贡献。B.K. 和 S.S. 致力于将 AlphaEvolve 应用于直接优化编译器生成的代码。M.E. 进行了消融实验。M.B.、A.N.、M.E.、S.S. 和 P.-S.H. 进行了

大部分代码审查工作。M.B.、E.D.、S.C.、N.V.、A.Z.W.、F.J.R.R.、M.E.、A.N.、B.K.、S.S.、A.M. 和 M.P.K. 撰写了论文，并得到了 A.S.、P.-S.H 和 P.K. 的贡献。N.V.、E.D.、M.E.、S.C.、A.N. 和 A.Z.W. 制作了图表。F.J.R.R.、A.M. 和 A.Z.W. 组装了随附的 Google Colab。S.N.、A.D. 和 P.K. 为这项工作的多个方面提供了建议和支持。M.B.、A.N.、N.V. 和 G.H. 协调了团队。P.K. 监督并协调了研究项目。

通讯作者。 Matej Balog、Alexander Novikov 和 Pushmeet Kohli。

参考文献

- [1] J. Abramson, J. Adler, J. Dunger, R. Evans, T. Green, A. Pritzel, O. Ronneberger, L. Willmore, A. J. Ballard, J. Bambrick, et al. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630(8016):493–500, 2024.
- [2] V. Aglietti, I. Ktena, J. Schrouff, E. Sgouritsa, F. J. R. Ruiz, A. Malek, A. Bellot, and S. Chiappa. FunBO: Discovering acquisition functions for Bayesian optimization with FunSearch. In *International Conference on Machine Learning*, 2025.
- [3] AlphaProof and AlphaGeometry teams. AI achieves silver-medal standard solving International Mathematical Olympiad problems, 2024. URL <https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level>.
- [4] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- [5] W. Banzhaf, P. Nordin, R. E. Keller, and F. D. Francone. *Genetic Programming: An Introduction on the Automatic Evolution of computer programs and its Applications*. The Morgan Kaufmann Series in Artificial Intelligence, 1998.
- [6] J. Bausch, A. W. Senior, F. J. H. Heras, T. Edlich, A. Davies, M. Newman, C. Jones, K. Satzinger, M. Y. Niu, S. Blackwell, G. Holland, D. Kafri, J. Atalaya, C. Gidney, D. Hassabis, S. Boixo, H. Neven, and P. Kohli. Learning high-accuracy error decoding for quantum processors. *Nature*, 635(8040):834–840, 2024. doi: 10.1038/s41586-024-08148-8.
- [7] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. doi: 10.1038/s41586-023-06792-0.
- [8] P. Boyvalenkov, S. Dodunekov, and O. Musin. A survey on the kissing numbers. *Serdica Math. J.*, 38(4):507–522, 2012. ISSN 1310-6600.
- [9] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.

-
- [10] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. doi: 10.1038/s42256-024-00832-8.
- [11] A. M. Bran, T. A. Neukomm, D. P. Armstrong, Z. Jončev, and P. Schwaller. Chemical reasoning in LLMs unlocks steerable synthesis planning and reaction mechanism elucidation. In *arXiv preprint arXiv:2503.08537*, 2025.
- [12] M. Caldas Ramos, C. J. Collison, and A. D. White. A review of large language models and autonomous agents in chemistry. *Chemical Science*, 16:2514–2572, 2025. doi: 10.1039/D4SC03921A.
- [13] P. S. Castro, N. Tomasev, A. Anand, N. Sharma, R. Mohanta, A. Dev, K. Perlin, S. Jain, K. Levin, N. Éltető, W. Dabney, A. Novikov, G. C. Turner, M. K. Eckstein, N. D. Daw, K. J. Miller, and K. L. Stachenfeld. Discovering symbolic cognitive models from human and animal behavior. In *International Conference on Machine Learning*, 2025.
- [14] A. Chen, D. M. Dohan, and D. R. So. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, 2023.
- [15] T. Chen, B. Xu, and K. Devleker. Automating GPU kernel generation with DeepSeek-R1 and inference time scaling, 2025. URL <https://developer.nvidia.com/blog/automating-gpu-kernel-generation-with-deepseek-r1-and-inference-time-scaling>.
- [16] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le. Symbolic discovery of optimization algorithms. *Advances in Neural Information Processing Systems*, 2023.
- [17] A. Cloninger and S. Steinerberger. On suprema of autoconvolutions with an application to Sidon sets. *Proceedings of the American Mathematical Society*, 145(8):3191–3200, 2017.
- [18] H. Cohn and F. Gonçalves. An optimal uncertainty principle in twelve dimensions via modular forms. *Inventiones mathematicae*, 217:799–831, 2019.
- [19] K. M. Collins, A. Q. Jiang, S. Frieder, L. Wong, M. Zilka, U. Bhatt, T. Lukasiewicz, Y. Wu, J. B. Tenenbaum, W. Hart, et al. Evaluating language models for mathematics through interactions. *Proceedings of the National Academy of Sciences*, 121(24):e2318124121, 2024.
- [20] K. D. Cooper, D. Subramanian, and L. Torczon. Adaptive optimizing compilers for the 21st century. *The Journal of Supercomputing*, 23:7–22, 2002.
- [21] M. Cranmer. Interpretable machine learning for science with pysr and symbolicregression. *jl. arXiv preprint arXiv:2305.01582*, 2023.
- [22] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
-

- [23] A. Davies, P. Veličković, L. Buesing, S. Blackwell, D. Zheng, N. Tomašev, R. Tanburn, P. Battaglia, C. Blundell, A. Juhász, M. Lackenby, G. Williamson, D. Hassabis, and P. Kohli. Advancing mathematics by guiding human intuition with AI. *Nature*, 600(7887):70–74, 2021. doi: 10.1038/s41586-021-04086-x.
- [24] J. S. Ellenberg, C. S. Fraser-Taliente, T. R. Harvey, K. Srivastava, and A. V. Sutherland. Generative modelling for mathematical discovery. *arXiv preprint arXiv:2503.11061*, 2025.
- [25] P. Erdős. Some remarks on number theory. *Riveon Lematematika*, 9:45–48, 1955.
- [26] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatin, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, D. Silver, D. Hassabis, and P. Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. doi: 10.1038/s41586-022-05172-4.
- [27] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel. Prompt-breeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- [28] N. Ferruz and B. Höcker. Controllable protein design with language models. *Nature Machine Intelligence*, 4(6):521–532, 2022.
- [29] E. Friedman. Erich’s Packing Center. <https://erich-friedman.github.io/packing/>, 2025. Accessed: 2025-04-22.
- [30] F. Frohnert, X. Gu, M. Krenn, and E. van Nieuwenburg. Discovering emergent connections in quantum physics research via dynamic word embeddings. *Machine Learning: Science and Technology*, 6(1):015029, 2025. doi: 10.1088/2632-2153/ad00a.
- [31] M. Ganzhinov. Highly symmetric lines. In *arXiv preprint arXiv:2207.08266v1*, 2022.
- [32] Gemini team. Gemini 2.5: Our most intelligent AI model, 2025. URL <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025>.
- [33] F. Gonçalves, D. O. e Silva, and S. Steinerberger. Hermite polynomials, linear flows on the torus, and an uncertainty principle for roots. *Journal of Mathematical Analysis and Applications*, 451(2):678–711, 2017.
- [34] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, A. Palepu, P. Sirkovic, A. Myaskovsky, F. Weissenberger, K. Rong, R. Tanno, K. Saab, D. Popovici, J. Blum, F. Zhang, K. Chou, A. Hassidim, B. Gokturk, A. Vahdat, P. Kohli, Y. Matias, A. Carroll, K. Kulkarni, N. Tomasev, Y. Guan, V. Dhillon, E. D. Vaishnav, B. Lee, T. R. D. Costa, J. R. Penadés, G. Peltz, Y. Xu, A. Pawlosky, A. Karthikesalingam, and V. Natarajan. Towards an AI co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.
- [35] A. Grayeli, A. Sehgal, O. Costilla Reyes, M. Cranmer, and S. Chaudhuri. Symbolic regression with a learned concept library. *Advances in Neural Information Processing Systems*, 37:44678–44709, 2024.

- [36] M. Gridach, J. Nanavati, C. Mack, K. Z. E. Abidine, and L. Mendes. Agentic AI for scientific discovery: A survey of progress, challenges, and future directions. In *ICLR Workshop: Towards Agentic AI for Science: Hypothesis Generation, Comprehension, Quantification, and Validation*, 2025.
- [37] X. Gu and M. Krenn. Interesting scientific idea generation using knowledge graphs and LLMs: Evaluations with 100 research group leaders. In *arXiv preprint arXiv:2405.17044*, 2024.
- [38] S. Guo, A. H. Shariatmadari, G. Xiong, and A. Zhang. Embracing foundation models for advancing scientific discovery. In *Proceedings of the IEEE International Conference on Big Data*, pages 1746–1755, 2024. doi: 10.1109/bigdata62323.2024.10825618.
- [39] K. Gyarmati, F. Hennecart, and I. Z. Ruzsa. Sums and differences of finite sets. *Functiones et Approximatio Commentarii Mathematici*, 37(1):175–186, 2007.
- [40] J. K. Haugland. The minimum overlap problem revisited. *arXiv preprint arXiv:1609.08000*, 2016.
- [41] E. Hemberg, S. Moskal, and U.-M. O’Reilly. Evolving code with a large language model. *Genetic Programming and Evolvable Machines*, 25(2):21, 2024. doi: 10.1007/s10710-024-09494-2.
- [42] J. E. Hopcroft and L. R. Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM J. Appl. Math.*, 20(1):30–36, Jan. 1971. ISSN 0036-1399. doi: 10.1137/0120004.
- [43] K. Huang, Y. Qu, H. Cousins, W. A. Johnson, D. Yin, M. Shah, D. Zhou, R. Altman, M. Wang, and L. Cong. CRISPR-GPT: An LLM agent for automated design of gene-editing experiments. In *arXiv preprint arXiv:2404.18021*, 2024.
- [44] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2): 1–55, 2025.
- [45] S. Jia, C. Zhang, and V. Fung. LLMatDesign: Autonomous materials discovery with large language models. In *arXiv preprint arXiv:2406.13163*, 2024.
- [46] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873): 583–589, 2021. doi: 10.1038/s41586-021-03819-2.
- [47] M. Kauers and J. Moosbauer. Flip graphs for matrix multiplication. In *Proceedings of the 2023 International Symposium on Symbolic and Algebraic Computation*, pages 381–388, 2023.

- [48] M. Kauers and J. Moosbauer. Some new non-commutative matrix multiplication algorithms of size $(n, m, 6)$. *ACM Commun. Comput. Algebra*, 58(1):1–11, Jan. 2025. ISSN 1932-2232. doi: 10.1145/3712020.3712021.
- [49] M. Kauers and I. Wood. Consequences of the Moosbauer-Poole algorithms. *arXiv preprint arXiv:2505.05896*, 2025.
- [50] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [51] J. R. Koza. Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4(2):87–112, 1994. doi: 10.1007/BF00175355.
- [52] J. D. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bulletin of the American Mathematical Society*, 82(1):126 – 128, 1976.
- [53] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirnsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Merose, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed 和 P. Battaglia. 学习熟练的中期全球天气预报. *Science*, 382(6677):1416 – 1421, 2023. doi: 10.1126/science.adi2336. [54] W. B. Langdon 和 R. Poli. 遗传编程基础. Springer Science & Business Media, 2013.
- [55] R. Lange, Y. Tian 和 Y. Tang. 大语言模型作为进化策略. 载于《遗传与进化计算会议论文集》, GECCO’24 论文集, 第 579 – 582 页. 美国计算机协会, 2024 年. doi: 10.1145/3638530.3654238.
- [56] R. T. Lange, A. Prasad, Q. Sun, M. Faldor, Y. Tang 和 D. Ha. AI CUDA 工程师: 智能体式 CUDA 核函数发现、优化与组合. 技术报告, Sakana AI, 2025 年 2 月. [57] J. Lehman, J. Gordon, S. Jain, K. Ndousse, C. Yeh 和 K. O. Stanley. 通过大模型进行进化. 载于《进化机器学习手册》, 第 331 – 366 页. Springer, 2023 年.
- [58] J. Lehman, J. Gordon, S. Jain, K. Ndousse, C. Yeh 和 K. O. Stanley. 通过大模型进行进化, 第 331 – 366 页. 施普林格·自然新加坡, 2024 年. doi: 10.1007/978-981-99-3814-8_11.
- [59] P.-H. Li, Y.-Y. Sun, H.-F. Juan, C.-Y. Chen, H.-K. Tsai, and J.-H. Huang. 一种基于文献的疾病-基因关联预测大语言模型框架. *生物信息学简报*, 26(1):bbaf070, 02 2025. ISSN 1477-4054. doi: 10.1093/bib/bbaf070. [60] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d’Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz,

E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu 和 O. Vinyals. 竞争-利用 AlphaCode 进行水平代码生成。《科学》, 378(6624):1092 – 1097, 2022 年。doi: 10.1126/science.abq1158。[61] H. Lin, M. Maas, M. Roquemoire, A. Hasanzadeh, F. Lewis, Y. Simonson, T.-W. Yang, A. Yazdanbakhsh, D. Altinbüken, F. Papa, 等。ECO: 一种面向仓库规模计算机的 LLM 驱动高效代码优化器。arXiv 预印本 arXiv:2503.15669, 2025 年。[62] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smetanin, R. Verkuil, O. Kabeli,

Y. Shmueli, A. dos Santos Costa, M. Fazel-Zarandi, T. Sercu, S. Candido, 和 A. Rives. 利用语言模型进行蛋白质原子级结构的进化尺度预测。Science, 379(6637):1123 – 1130, 2023. doi: 10.1126/science.ade2574。[63] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, 和 Q. Zhang. 启发式算法的进化: 利用大语言模型实现高效自动算法设计。arXiv 预印本 arXiv:2401.02051, 2024。[64] F. Luo, J. Zhang, Q. Wang, 和 C. Yang. 利用大语言模型中的提示工程加速化学研究。ACS Central Science, 2025. doi: 10.1021/acscentsci.4c01935。[65] Z. Luo, Z. Yang, Z. Xu, W. Yang, 和 X. Du. LLM4SR: 大语言模型在科学研究中的应用综述。见 arXiv 预印本 arXiv:2501.04306, 2025。[66] H. Ma, A. Narayanaswamy, P. Riley, 和 L. Li. 进化符号密度泛函。Science Advances, 8(36):eabq0279, 2022. doi: 10.1126/sciadv.abq0279。[67] A. Madani, B. Krause, E. R. Greene, S. Subramanian, B. P. Mohr, J. M. Holton, J. L. Olmos, C. Xiong, Z. Z. Sun, R. Socher, J. S. Fraser, 和 N. Naik. 大语言模型生成跨家族的功能性蛋白质序列。Nature Biotechnology, 41(8):1099 – 1106, 2023 年 8 月。ISSN 1087-0156. doi: 10.1038/s41587-022-01618-2。[68] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent,

伊克巴尔、莱斯皮奥、埃亨、科普、米利金、加夫尼、埃尔斯特、布罗希尔, 甘布尔、米兰、董、黄、塞姆吉尔、巴雷卡坦、李宇、曼达内, T. Hubert, J. Schrittwieser, D. Hassabis, P. Kohli, M. Riedmiller, O. Vinyals, 和 D. Silver. 使用深度强化学习发现的更快的排序算法。《自然》, 618(7964):257 – 263, 2023 年。doi: 10.1038/s41586-023-06004-9。[69] 马萨林。超级优化器——最小程序之探究。载于卡茨与弗里曼主编, 《第二届编程语言与操作系统架构支持国际会议论文集》(ASPLOS II), 美国加利福尼亚州帕洛阿尔托, 1987 年 10 月 5-8 日, 第 122 – 126 页。ACM 出版社, 1987 年。doi: 10.1145/36206.36194。[70] 马托尔奇与维努埃萨。自卷积上确界的改进界。《数学分析与应用杂志》, 372(2):439 – 447, 2010 年。[71] 米雷特与克里希南。大语言模型是否已准备好用于现实世界的材料发现? 载于 arXiv 预印本 arXiv:2402.05200, 2024 年。

- [72] J. Moosbauer 和 M. Poole。具有对称性的翻转图与新的矩阵乘法方案。arXiv 预印本 arXiv:2502.04514, 2025 年。[73] C. Morris、M. Jurado 和 J. Zutty。大语言模型引导的进化——模型推进模型的自动化。载于《遗传与进化计算会议论文集》，第 377–384 页，2024 年。[74] J.-B. Mouret 和 J. Clune。通过映射精英照亮搜索空间。arXiv 预印本 arXiv:1504.04909, 2015 年。
- [75] V. Naumov、D. Zagirova、S. Lin、Y. Xie、W. Gou、A. Urban、N. Tikhonova、K. Alawi、M. Durymanov、F. Galkin、S. Chen、D. Sidorenko、M. Korzinkin、M. Scheibye-Knudsen、A. Aspuru-Guzik、E. Izumchenko、D. Gennert、F. W. Pun、M. Zhang、P. Kamya、A. Aliper、F. Ren 和 A. Zhavoronkov。DORA 人工智能科学家：用于科学探索发现和自动报告生成的多智能体虚拟研究团队。载于 bioRxiv 预印本：10.1101/2025.03.06.641840。冷泉港实验室，2025 年。doi: 10.1101/2025.03.06.641840。
- [76] OpenAI。介绍 OpenAI o3 和 o4-mini, 2025 年。网址 <https://openai.com/index/introducing-o3-and-o4-mini/>。[77] OpenXLA。XLA: Python+NumPy 程序的可组合变换。网址 <https://github.com/openxla/xla>。[78] H. Pan、N. Mudur、W. Taranto、M. Tikhonovskaya、S. Venugopalan、Y. Bahri、M. P. Brenner 和 E.-A. Kim。利用大语言模型进行量子多体物理计算。《通信物理》，8(1):49, 2025 年。doi: 10.1038/s42005-025-01956-y。[79] D. Pantiukhin、B. Shapkin、I. Kuznetsov、A. A. Jost 和 N. Koldunov。通过多智能体大语言模型系统加速地球科学发现。载于 arXiv 预印本 arXiv:2503.05854, 2025 年。[80] Z. Rasheed、M. Waseem、A. Ahmad、K.-K. Kemell、W. Xiaofeng、A. N. Duc 和 P. Abrahamsson。大语言模型能否充当数据分析师？一种用于定性数据分析的多智能体辅助方法。arXiv 预印本 arXiv:2402.01386, 2024 年。[81] S. Ren、P. Jian、Z. Ren、C. Leng、C. Xie 和 J. Zhang。迈向科学智能：基于大语言模型的科学智能体综述。载于 arXiv 预印本 arXiv:2503.24047, 2025 年。[82] A. Rives、J. Meier、T. Sercu、S. Goyal、Z. Lin、J. Liu、D. Guo、M. Ott、C. L. Zitnick、J. Ma 和 R. Fergus。将无监督学习扩展到 2.5 亿个蛋白质序列，生物结构与功能随之涌现。《美国国家科学院院刊》，118(15):e2016239118, 2021 年。doi: 10.1073/pnas.2016239118。[83] B. Romera-Paredes、M. Barekatain、A. Novikov、M. Balog、M. P. Kumar、E. Dupont、

F. J. R. Ruiz、J. Ellenberg、P. Wang、O. Fawzi、P. Kohli 和 A. Fawzi。数学大语言模型程序搜索的发现。《自然》，625(7995): 468–475, 2023 年。doi: 10.1038/s41586-023-06924-6。

[84] F. J. R. 鲁伊斯, T. 拉科宁, J. 鲍施, M. 巴洛格, M. 巴雷卡坦, F. J. H. 赫拉斯,

ichanetzidis 与 P. Kohli。利用 AlphaTensor 进行量子电路优化。《自然·机器学习》, 7(3):374 – 385, 2025 年。doi: 10.1038/s42256-025-01001-1。[85] O. A. Sarumi 与 D. Heider。大语言模型及其在生物信息学中的应用。《计算与结构生物技术杂志》, 23:3498 – 3505, 2024 年。ISSN 2001-0370。doi: <https://doi.org/10.1016/j.csbj.2024.09.031>。[86] E. Schkufza、R. Sharma 与 A. Aiken。随机超优化。载于 V. Sarkar 与

R. Bodik, 编辑, 《编程语言与操作系统的体系结构支持》系统, ASPLOS 2013, 美国德克萨斯州休斯顿, 2013 年 3 月 16 日至 20 日, 第 305 – 316 页。ACM, 2013 年。doi: 10.1145/2451116.2451150。[87] M. Schmidt 和 H. Lipson。从实验数据中提炼自由形式自然法则。《科学》, 324(5923):81 – 85, 2009 年。doi: 10.1126/science.1165893。[88] N. Shinn、F. Cassano、A. Gopinath、K. Narasimhan 和 S. Yao。Reflexion: 具有言语强化学习的语言智能体。《神经信息处理系统进展》, 36:8634 – 8652, 2023 年。[89] P. Shojaei、K. Meidani、S. Gupta、A. B. Farimani 和 C. K. Reddy。LLM-SR: 通过大语言模型编程进行科学方程发现。载于《国际学习表征会议》, 2025 年。[90] C. Si、D. Yang 和 T. Hashimoto。大语言模型能否产生新颖的研究想法? 一项涉及 100 多位自然语言处理研究人员的大规模人类研究。载于《国际学习表征会议》, 2025 年。[91] A. Smirnov。矩阵乘法问题的若干双线性算法 $\langle 3, P, Q \rangle$ 。 https://www.researchgate.net/publication/350897049_Several_Bilinear_Algorithms_for_Matrix_Multiplication_Problems_3_P_Q, 2021 年 4 月。[92] A. Smirnov。矩阵乘法的双线性算法 $\langle 4 \times 4 \times 9; 104 \rangle$ 。布伦特系统的一个不可约无理解? https://www.researchgate.net/publication/364167198_Bilinear_Algorithm_for_Matrix_Multiplication_4x4x9_104_An_irreducibly_irrational_solution_of_the_Brent_system, 2022 年 10 月。[93] A. V. Smirnov。矩阵乘法的双线性复杂度与实用算法。《计算数学与数学物理》, 53(12):1781 – 1795, 2013 年。[94] Z. Song、M. Ju、C. Ren、Q. Li、C. Li、Q. Zhou 和 J. Wang。LLM-Feynman: 利用大语言模型进行通用科学公式与理论发现。载于 arXiv 预印本 arXiv:2503.06512, 2025 年。[95] V. Strassen。高斯消元法并非最优。《数值数学》, 13(4):354 – 356, 1969 年。

- [96] A. Surina、A. Mansouri、L. Quaedvlieg、A. Seddas、M. Viazovska、E. Abbe 和 C. Gulcehre。利用大语言模型进行算法发现：进化搜索与强化学习的结合。arXiv 预印本 arXiv:2504.05108, 2025 年。[97] R. Tanese。用于函数优化的分布式遗传算法。密歇根大学, 1989 年。[98] A. Thakur、G. Tsoukalas、Y. Wen、J. Xin 和 S. Chaudhuri。用于形式定理证明的上下文学习智能体。语言模型会议, 2024 年。[99] T. H. Trinh、Y. Wu、Q. V. Le、H. He 和 T. Luong。无需人类演示解决奥林匹克几何问题。《自然》, 625(7995):476 – 482, 2024 年。[100] A. Vaswani、N. Shazeer、N. Parmar、J. Uszkoreit、L. Jones、A. N. Gomez、L. Kaiser 和 I. Polosukhin。注意力机制就是你所需要的一切。神经信息处理系统进展, 2017 年。[101] P. Veličković、A. Vitvitskyi、L. Markeeva、B. Ibarz、L. Buesing、M. Balog 和 A. Novikov。在组合竞技编程中放大人类表现。2024 年。[102] A. Verma、L. Pedrosa、M. Korupolu、D. Oppenheimer、E. Tune 和 J. Wilkes。使用 Borg 在谷歌进行大规模集群管理。第十届欧洲计算机系统会议论文集, EuroSys '15, 美国纽约州纽约市, 2015 年。美国计算机协会。ISBN 9781450332385。doi: 10.1145/2741948.2741964。[103] S. Verma、A. Goyal、A. Mathur、A. Anand 和 S. Ranu。GRAIL: 利用大语言模型生成代码进行图编辑距离和节点对齐。国际机器学习会议, 2025 年。[104] C. Vinuesa del Rio。广义 Sidon 集。博士论文, 马德里自治大学, 2010 年。[105] H. Wang、M. Skreta、C. T. Ser、W. Gao、L. Kong、F. Strieth-Kalthoff、C. Duan、Y. Zhuang,

Y. Yu, Y. Zhu, Y. Du, A. Aspuru-Guzik, K. Neklyudov 和 C. Zhang。高效的进化-利用大语言模型在化学空间中进行进化搜索。载于国际学习表征会议, 2025 年。[106] Q. Wang, D. Downey, H. Ji 和 T. Hope。SciMON: 面向新颖性的科学启发机器。载于第 62 届计算语言学协会年会论文集 (第一卷: 长文), 第 279 – 299 页, 泰国曼谷, 2024 年。计算语言学协会。doi: 10.18653/v1/2024.acl-long.18。[107] E. P. White。埃尔德什最小重叠问题的新界。算术学报, 208:235 – 255, 2023 年。[108] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu,

A. H. 阿瓦达拉、R. W. 怀特、D. 伯格和 C. 王。AutoGen: 通过多智能体对话实现下一代大语言模型应用。见 arXiv 预印本 arXiv:2308.08155, 2023 年。

- [109] Y. 夏、P. 金、S. 谢、L. 何、C. 曹、R. 罗、G. 刘、Y. 王、Z. 刘、Y.-J. 陈、Z. 郭、Y. 白、P. 邓、Y. 敏、Z. 卢、H. 郝、H. 杨、J. 李、C. 刘、J. 张、J. 朱、R. 毕、K. 吴、W. 张、K. 高、Q. 裴、Q. 王、X. 刘、Y. 李、H. 朱、Y. 卢、M. 马、Z. 王、T. 谢、K. 马齐亚尔兹、M. 塞格勒、Z. 杨、Z. 陈、Y. 施、S. 郑、L. 吴、C. 胡、P. 戴、T.-Y. Liu, H. Liu, and T. Qin. 自然语言模型：解读自然语言以促进科学发现。见 arXiv 预印本 arXiv:2502.07527v2, 2025. [110] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar. Leandojo: 使用检索增强语言模型进行定理证明。神经信息处理系统进展, 36:21573 – 21612, 2023. [111] K. Yang, G. Poesia, J. He, W. Li, K. Lauter, S. Chaudhuri, and D. Song. 形式化数学推理：人工智能的新前沿。arXiv 预印本 arXiv:2412.16075, 2024. [112] Z. Yang, X. Du, J. Li, J. Zheng, S. Poria, and E. Cambria. 用于自动化开放领域科学假设发现的大语言模型。见计算语言学协会发现：ACL 2024, 第 13545 – 13565 页。计算语言学协会, 2024. doi: 10.18653/v1/2024.findings-acl.804. [113] Z. Yang, W. Liu, B. Gao, T. Xie, Y. Li, W. Ouyang, S. Poria, E. Cambria, and D. Zhou. MOOSE-Chem: 用于重新发现未见化学科学假设的大语言模型。见国际学习表征会议, 2025. [114] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: 在语言模型中协同推理与行动。见国际学习表征会议 (ICLR), 2023. [115] S. Yao, F. Liu, X. Lin, Z. Lu, Z. Wang, and Q. Zhang. 使用大语言模型进行启发式多目标进化。见 AAAI 人工智能会议, 第 39 卷, 第 27144 – 27152 页, 2025. doi: 10.1609/aaai.v39i25.34922. [116] G. Ye, X. Cai, H. Lai, X. Wang, J. Huang, L. Wang, W. Liu, and X. Zeng. DrugAssist: 用于分子优化的大语言模型。见 arXiv 预印本 arXiv:2401.10334, 2023. [117] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song. ReEvo: 作为具有反思进化能力的超启发式大语言模型。见神经信息处理系统进展, 第 37 卷, 2024. [118] F. Zhang, S. Nguyen, Y. Mei, and M. Zhang. 生产调度的遗传编程。Springer, 2021. [119] H. Zhang, Y. Song, Z. Hou, S. Miret, and B. Liu. HoneyComb: 一个灵活的基于大语言模型的材料科学代理系统。见 Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, 编辑, 计算语言学协会发现：EMNLP 2024, 第 3369 – 3382 页。计算语言学协会, 2024 年 11 月。doi: 10.18653/v1/2024.findings-emnlp.192. [120] Y. Zhou, H. Liu, T. Srivastava, H. Mei, and C. Tan. 使用大语言模型进行假设生成。见 L. Peled-Cohen, N. Calderon, S. Lissak, and R. Reichart, 编辑,

第 1 届科学自然语言处理研讨会 (NLP4Science) 论文集, 第 117 – 139 页。计算语言学协会, 2024 年。doi: 10.18653/v1/2024.nlp4science-1.10。

A. 更快的矩阵乘法：完整结果

完整结果表。我们在表 3 中提供了 AlphaEvolve 获得的最佳秩。总体而言，我们在实验中考虑了 54 种矩阵乘法规模。这些规模大致代表 $\langle m, n, p \rangle$ ，其中 $2 < m, n < 5$ ，并对 p 设置了一些合理的截断。由于底层矩阵乘法张量的对称性，对于三个轴的任意排列存在等价算法，因此我们专注于排序后的规模 $m \leq n \leq p$ 。

在除两种规模外的所有考虑规模中，AlphaEvolve 发现的程序要么匹配要么超越已知最佳秩。有趣的是，在增大问题规模时我们遇到了一些困难：当我们在配备单个 GPU 加速器的评估器上以 1000 个随机种子运行发现的程序处理超过 $\langle 5.5.5 \rangle$ 的规模时，经常出现内存不足。因此，将我们的设置扩展到更大的矩阵规模需要进一步优化。

$\langle m, n, p \rangle$	best known [reference]	AlphaEvolve	$\langle m, n, p \rangle$	best known [reference]	AlphaEvolve	$\langle m, n, p \rangle$	best known [reference]	AlphaEvolve
$\langle 2, 2, 2 \rangle$	7 [95]	7	$\langle 2, 3, 6 \rangle$	30 [93]	30	$\langle 3, 4, 4 \rangle$	38 [93]	38
$\langle 2, 2, 3 \rangle$	11 [93]	11	$\langle 2, 3, 7 \rangle$	35 [93]	35	$\langle 3, 4, 5 \rangle$	47 [26]	47
$\langle 2, 2, 4 \rangle$	14 [93]	14	$\langle 2, 3, 8 \rangle$	40 [93]	40	$\langle 3, 4, 6 \rangle$	56 [48]	54
$\langle 2, 2, 5 \rangle$	18 [93]	18	$\langle 2, 3, 9 \rangle$	45 [93]	45	$\langle 3, 4, 7 \rangle$	66 [91]	63
$\langle 2, 2, 6 \rangle$	21 [93]	21	$\langle 2, 3, 10 \rangle$	50 [93]	50	$\langle 3, 4, 8 \rangle$	75 [91]	74
$\langle 2, 2, 7 \rangle$	25 [93]	25	$\langle 2, 4, 4 \rangle$	26 [93]	26	$\langle 3, 5, 5 \rangle$	58 [91]	58
$\langle 2, 2, 8 \rangle$	28 [93]	28	$\langle 2, 4, 5 \rangle$	33 [42]	32	$\langle 3, 5, 6 \rangle$	70 [48]	68
$\langle 2, 2, 9 \rangle$	32 [93]	32	$\langle 2, 4, 6 \rangle$	39 [93]	39	$\langle 3, 5, 7 \rangle$	82 [91]	80
$\langle 2, 2, 10 \rangle$	35 [93]	35	$\langle 2, 4, 7 \rangle$	46 [93]	45	$\langle 4, 4, 4 \rangle$	49 [95]	48
$\langle 2, 2, 11 \rangle$	39 [93]	39	$\langle 2, 4, 8 \rangle$	52 [93]	51	$\langle 4, 4, 5 \rangle$	62 [47]	61
$\langle 2, 2, 12 \rangle$	42 [93]	42	$\langle 2, 5, 5 \rangle$	40 [93]	40	$\langle 4, 4, 6 \rangle$	73 [48]	73
$\langle 2, 2, 13 \rangle$	46 [93]	46	$\langle 2, 5, 6 \rangle$	48 [93]	47	$\langle 4, 4, 7 \rangle$	87 [93, 95]	85
$\langle 2, 2, 14 \rangle$	49 [93]	49	$\langle 3, 3, 3 \rangle$	23 [52]	23	$\langle 4, 4, 8 \rangle$	98 [95]	96
$\langle 2, 2, 15 \rangle$	53 [93]	53	$\langle 3, 3, 4 \rangle$	29 [93]	29	$\langle 4, 4, 9 \rangle$	104 [92]	108
$\langle 2, 2, 16 \rangle$	56 [93]	56	$\langle 3, 3, 5 \rangle$	36 [93]	36	$\langle 4, 5, 5 \rangle$	76 [26]	76
$\langle 2, 3, 3 \rangle$	15 [93]	15	$\langle 3, 3, 6 \rangle$	40 [93]	40	$\langle 4, 5, 6 \rangle$	93 [48]	90
$\langle 2, 3, 4 \rangle$	20 [93]	20	$\langle 3, 3, 7 \rangle$	49 [93]	49	$\langle 5, 5, 5 \rangle$	93 [72]	93
$\langle 2, 3, 5 \rangle$	25 [93]	25	$\langle 3, 3, 8 \rangle$	55 [93]	55	$\langle 6, 6, 6 \rangle$	153 [72]	156

表 3 | 表 2 的完整版本，展示了 AlphaEvolve 在所有考虑参数下张量分解获得的最佳秩。在 54 个目标中，AlphaEvolve 在 38 个案例中匹配最新技术水平，在 14 个案例中超越（绿色），在 2 个案例中落后（红色）。在所有案例中，AlphaEvolve 都提供了精确算法，分解中使用整数或半整数项。对于 $\langle 3.4.7 \rangle$ 、 $\langle 4.4.4 \rangle$ 和 $\langle 4.4.8 \rangle$ ，AlphaEvolve 发现的算法使用复值乘法，可用于复数或实数矩阵的精确乘法。本表中展示的分解可在随附的 Google Colab 中找到。

注：并行工作 [49] 也为 $\langle 4.5.6 \rangle$ 找到了秩为 90 的算法。

图 4（左）的放大版本。在图 9a 至 9c 中，我们展示了图 4（左）的放大版本，该图对应于发现秩为 48 的分解的程序，该分解针对表示两个 4×4 矩阵相乘运算的三维张量。

```

1 @@ -45,9 +45,14 @@
2 # EVOLVE-BLOCK-START
3 def _get_optimizer(self) -> optax.GradientTransformation:
4     """Returns optimizer."""
5     - return optax.adam(self.hypers.learning_rate)
6     + return optax.adamw(
7     +     self.hypers.learning_rate, weight_decay=self.hypers.weight_decay
8     + )
9
10 def _get_init_fn(self) -> jax.nn.initializers.Initializer:
11     """Returns initializer function."""
12     - return initializers.normal(0.0, self.hypers.init_scale, jnp.complex64)
13     + # Initialize with a smaller scale to encourage finding low-rank
14     + # solutions.
15     + # Increase scale slightly for better exploration.
16     + scale = self.hypers.init_scale
17     + return initializers.normal(0 + 1j * 0, scale * 0.2, jnp.complex64)
18 @@ -80,6 +85,66 @@
19 # Gradient updates.
20 updates, opt_state = self.opt.update(grads, opt_state, decomposition)
21 decomposition = optax.apply_updates(decomposition, updates)
22 + # Add a small amount of gradient noise to help with exploration
23 + rng, g_noise_rng = jax.random.split(rng)
24 + decomposition = jax.tree_util.tree_map(
25 +     lambda x: x
26 +     + self.hypers.grad_noise_std * jax.random.normal(g_noise_rng, x.
27 + shape),
28 +     decomposition,
29 + )
30 +
31 + # Add noise to the decomposition parameters (exploration).
32 + _, noise_rng = jax.random.split(rng)
33 + noise_std = self._linear_schedule(
34 +     global_step, start=self.hypers.noise_std, end=0.0
35 + )
36 + decomposition = jax.tree_util.tree_map(
37 +     lambda x: x + noise_std * jax.random.normal(noise_rng, x.shape),
38 +     decomposition,
39 + )
40 +
41 + # Cyclical annealing for clipping threshold.
42 + cycle_length = 2000 # Number of steps per cycle
43 + cycle_progress = (
44 +     global_step % cycle_length
45 + ) / cycle_length # Normalized progress within the current cycle [0,
46 + 1)
47 +
48 + # Map cycle progress to a sinusoidal curve. Ranges from 0 to 1.
49 + clip_threshold_multiplier = (1 + jnp.cos(2 * jnp.pi * cycle_progress))
50 + / 2
51 +
52 + clip_threshold = self.hypers.clip_min + clip_threshold_multiplier * (
53 +     self.hypers.clip_max - self.hypers.clip_min
54 + )
55 +
56 + def soft_clip(x, threshold):
57 +     # Clipping the real and imaginary parts separately.
58 +     x_re = jnp.real(x)
59 +     x_im = jnp.imag(x)
60 +
61 +     x_re_clipped = jnp.where(
62 +         x_re > threshold, threshold + (x_re - threshold) * 0.1, x_re
63 +     )
64 +     x_re_clipped = jnp.where(
65 +         x_re_clipped < -threshold,
66 +         -threshold + (x_re_clipped + threshold) * 0.1,
67 +         x_re_clipped,
68 +     )

```

图 9a | 图 4 (左) 的放大版本, 给出了发现更快的 4×4 矩阵相乘算法的程序 (1/3).


```

66 +
67 +     x_im_clipped = jnp.where(
68 +         x_im > threshold, threshold + (x_im - threshold) * 0.1, x_im
69 +     )
70 +     x_im_clipped = jnp.where(
71 +         x_im_clipped < -threshold,
72 +         -threshold + (x_im_clipped + threshold) * 0.1,
73 +         x_im_clipped,
74 +     )
75 +
76 +     return x_re_clipped + 1j * x_im_clipped
77 +
78 +     decomposition = jax.tree_util.tree_map(
79 +         lambda x: soft_clip(x, clip_threshold), decomposition
80 +     )
81 +
82 +     return decomposition, opt_state, loss
83
84 def _loss_fn(
85 @@ -91,13 +156,86 @@
86     """Computes (batched) loss on learned decomposition."""
87     # Compute reconstruction loss.
88     rec_tensor = self._decomposition_to_tensor(decomposition) # (B, N, M,
89     P)
90 +
91 +     # Add noise to the target tensor (robustness).
92 +     rng, noise_rng = jax.random.split(rng)
93 +     target_noise = self.hypers.target_noise_std * jax.random.normal(
94 +         noise_rng, self.target_tensor.shape
95 +     )
96 +     noisy_target_tensor = self.target_tensor + target_noise
97 +
98 +     # Hallucination loss (encourages exploration by randomly replacing
99 +     values)
100 +     hallucination_prob = self.hypers.hallucination_prob
101 +     hallucination_scale = self.hypers.hallucination_scale
102 +
103 +     def hallucinate(x, hallucination_rng):
104 +         mask = jax.random.bernoulli(hallucination_rng, p=hallucination_prob)
105 +         noise = hallucination_scale * jax.random.normal(
106 +             hallucination_rng, x.shape
107 +         )
108 +         return jnp.where(mask, noise, x)
109 +
110 +     _, factor_rng = jax.random.split(rng)
111 +     decomposition = jax.tree_util.tree_map(
112 +         lambda x: hallucinate(x, jax.random.split(factor_rng)[0]),
113 +         decomposition,
114 +     )
115 +
116 +     # 向`target_tensor`添加一个批次维度`以确保正确的广播。
117 +
118 +     loss as the L2 reconstruction error.
119 +     rec_loss = l2_loss_complex(self.target_tensor[None, ...], rec_tensor)
120 +     rec_loss = l2_loss_complex(noisy_target_tensor[None, ...], rec_tensor)
121 +
122 +     # We must return a real-valued loss.
123 +     return jnp.real(rec_loss)
124 +
125 +     # Discretization loss (encourage entries to be multiples of 1/2 or
126 +     integer).
127 +     def dist_to_half_ints(x):
128 +         x_re = jnp.real(x)
129 +         x_im = jnp.imag(x)
130 +         return jnp.minimum(
131 +             jnp.abs(x_re - jnp.round(x_re * 2) / 2),
132 +             jnp.abs(x_im - jnp.round(x_im * 2) / 2),
133 +         )

```

图 9b | 图 4 (左) 的放大版本, 给出了发现更快的 4×4 矩阵相乘算法的程序 ($\frac{2}{3}$).

```

131 +     def dist_to_ints(x):
132 +         return jnp.abs(x - jnp.round(x))
133 +
134 +     discretization_loss = 0.0
135 +     for factor in decomposition:
136 +         discretization_loss += jnp.mean(dist_to_half_ints(factor))
137 +         discretization_loss += jnp.mean(dist_to_ints(factor))
138 +
139 +     discretization_loss /= (
140 +         len(decomposition) * 2
141 +     ) # average across all factors and loss components
142 +
143 +     discretization_weight = self._linear_schedule(
144 +         global_step, start=0.0, end=self.hypers.discretization_weight
145 +     )
146 +
147 +     # Cosine annealing for half-integer loss.
148 +     cycle_length = self.config.training_steps // 4 # Number of steps per
cycle
149 +     cycle_progress = (
150 +         global_step % cycle_length
151 +     ) / cycle_length # Normalized progress within the current cycle [0,
1)
152 +     half_int_multiplier = (1 + jnp.cos(jnp.pi * cycle_progress)) / 2
153 +     half_int_multiplier = (
154 +         1 - self.hypers.half_int_start
155 +     ) * half_int_multiplier + self.hypers.half_int_start
156 +
157 +     total_loss = (
158 +         rec_loss
159 +         + discretization_weight * discretization_loss *
half_int_multiplier
160 +     )
161 +
162 +     # Add penalty for large values (stability).
163 +     large_value_penalty = 0.0
164 +     for factor in decomposition:
165 +         large_value_penalty += jnp.mean(jnp.abs(factor) ** 2)
166 +     large_value_penalty /= len(decomposition)
167 +     total_loss += self.hypers.large_value_penalty_weight *
large_value_penalty
168 +
169 +     return jnp.real(total_loss)
170 +
171
172 def l2_loss_complex(x: jnp.ndarray, y: jnp.ndarray) -> jnp.ndarray:
173     """Elementwise L2 loss for complex numbers."""
174     @jit -117,6 +255,18 @jit
175     return hyper.zipit([
176 -         hyper.uniform('init_scale', hyper.interval(0.2, 1.5)),
177 -         hyper.uniform('learning_rate', hyper.interval(0.05, 0.3)),
178 +         hyper.uniform('init_scale', hyper.interval(0.1, 1.0)),
179 +         hyper.uniform('learning_rate', hyper.interval(0.01, 0.2)),
180 +         hyper.uniform('discretization_weight', hyper.interval(0.0, 0.1)),
181 +         hyper.uniform('hallucination_prob', hyper.interval(0.0, 0.2)),
182 +         hyper.uniform('hallucination_scale', hyper.interval(0.0, 0.2)),
183 +         hyper.uniform('noise_std', hyper.interval(0.0, 0.01)),
184 +         hyper.uniform('target_noise_std', hyper.interval(0.0, 0.01)),
185 +         hyper.uniform('weight_decay', hyper.interval(0.00001, 0.001)),
186 +         hyper.uniform('clip_min', hyper.interval(0.0, 0.5)),
187 +         hyper.uniform('clip_max', hyper.interval(1.0, 3.0)),
188 +         hyper.uniform('large_value_penalty_weight', hyper.interval(0.0,
0.01)),
189 +         # Add noise to the gradient to aid in exploration.
190 +         hyper.uniform('grad_noise_std', hyper.interval(0.0, 0.001)),
191 +         hyper.uniform('half_int_start', hyper.interval(0.0, 1.0)),
192     ])
193     # EVOLVE-BLOCK-END

```

图 9c | 图 4 (左) 的放大版本, 给出了发现更快的 4×4 矩阵相乘算法的程序 (3/3)。这里 hyper 是一个用户提供的用于生成超参数扫描的库。

B. AlphaEvolve 的数学发现详情

本节报告的所有构造的数据和验证代码均可在随附的 Google Colab 中找到。

B.1. 第一个自相关不等式 对于任意函数 $f: \mathbb{R} \rightarrow \mathbb{R}$, 定义 f 的自卷积为

$$f * f(t) := \int_{\mathbb{R}} f(t-x)f(x) dx.$$

设 C_1 表示满足以下条件的最大常数

$$\max_{-1/2 \leq t \leq 1/2} f * f(t) \geq C_1 \left(\int_{-1/4}^{1/4} f(x) dx \right)^2 \quad (1)$$

对所有非负 $f: \mathbb{R} \rightarrow \mathbb{R}$ 成立。该问题源于加法组合学, 与西顿集的大小有关。目前已知

$$1.28 \leq C_1 \leq 1.5098,$$

下界在 [17] 中取得, 上界在 [70] 中通过阶跃函数构造取得。AlphaEvolve 在 $[-1/4, 1/4]$ 上找到了一个具有 600 个等距区间的阶跃函数, 给出了略优的上界 $C_1 \leq 1.5053$.

B.2. 第二个自相关不等式 设 C_2 为满足 $\|f * f\|_2^2 \leq C_2 \|f\|_1 \|f\|_\infty$ 对所有非负 $f: \mathbb{R} \rightarrow \mathbb{R}$ 成立的最小常数。已知

$$0.88922 \leq C_2 \leq 1$$

下界来自阶跃函数构造 [70]。AlphaEvolve 在 $[-1/4, 1/4]$ 上找到了一个具有 50 个等距区间的阶跃函数, 给出了略优的结果 lower bound $0.8962 \leq C_2$.

B.3. 第三自相关不等式 设 C_3 为满足以下条件的最大常数:

$$\max_{-1/2 \leq t \leq 1/2} |f * f(t)| \geq C_3 \left(\int_{-1/4}^{1/4} f(x) dx \right)^2$$

对任意函数 $f: \mathbb{R} \rightarrow \mathbb{R}$. 显然 $C_3 < C_1$, 因为我们现在允许 f 取正值和负值。存在一个阶跃函数给出上界 $C_3 < 1.45810$ [104, 第 75 页]。AlphaEvolve 在 $[-1/4, 1/4]$ 上找到了一个具有 400 个等距区间的阶跃函数, 给出了略好的上界 $C_3 \leq 1.4557$.

B.4. 一个不确定性不等式 给定函数 $f: \mathbb{R} \rightarrow \mathbb{R}$, 定义傅里叶变换

$$\hat{f}(\xi) := \int_{\mathbb{R}} f(x) e^{-2\pi i x \xi} dx \text{ 且}$$

$$A(f) := \inf\{r > 0 : f(x) \geq 0 \text{ for all } |x| \geq r\}.$$

设 C_4 为使得下式成立的最大常数:

$$A(f)A(\hat{f}) \geq C_4$$

对所有满足 $\max(f(0), \hat{f}(0)) < 0$ 的偶函数 f 。已知 [33]

$$0.2025 \leq C_4 \leq 0.3523.$$

(论文中上界写为 0.353, 但将其解四舍五入到第四位小数得到 0.3523)。我们利用与 [33] 中类似的线性组合, 但使用由 AlphaEvolve 找到的精细常数, 将上界改进为 $C_4 < 0.3521$ 。

为了获得 C_4 的上界, 构造一个满足条件的特定“测试函数” f , 并计算该函数的值 $A(f)A(\hat{f})$, 从而提供上界 $C_4 \leq A(f)A(\hat{f})$ 。按照 [33] 中的方法, 测试函数的形式为, 其中 $P(x)$ 是一个偶多项式, 构造为埃尔米特多项式 $H_{4k}(x)$ 的线性组合。这种形式特别有用, 因为 $H_n(x)e^{-\pi x^2}$ 的傅里叶变换是 $i^n H_n(\xi)e^{-\pi \xi^2}$ 。对于偶多项式 $P(x) = \sum c_{4k} H_{4k}(x)$, 傅里叶变换与 $P(x)$ 的最大正根相关, 而 $\frac{P}{A(\hat{f})}$ 与 $P(\xi)$ 的最大正根相关。具体地, 如果对于大的 $|x|$, $A(f)$ 有 $P(x) > 0$, 则 $P(x)$ 的最大正根为, 且 $\frac{P}{A(\hat{f})}$ 是 $P(\xi)$ 的最大正根, 这意味着 $\frac{P}{A(\hat{f})} = A(f)$ 。不等式变为 $C_4 \leq (A(f))^2$ 。

$$f(x) = P(x)e^{-\pi x^2}$$

$$\text{of } f(x) \text{ is } f(\xi) = \sum c_{4k} i^{4k} H_{4k}(\xi) e^{-\pi \xi^2} = \left(\sum c_{4k} H_{4k}(\xi) \right) e^{-\pi \xi^2} = P(\xi) e^{-\pi \xi^2} \quad \text{Thus, } A(f)$$

该方法涉及为多项式 $P(x) = c_0 H_0(x) + c_1 H_4(x) + c_2 H_8(x) + \dots$ 寻找系数 $c_0, c_1, c_2, \dots$, 使得 $P(x)$ 满足某些约束 (与 $\frac{P}{A(\hat{f})}$ 相关且对大的 $|x|$ 为正), 并最小化 $P(x)$ 的最大正根。在 $f(0) < 0, \hat{f}(0) < 0$

我们的方法中, 多项式 $P(x)$ 被构造使得 $P(0) = 0$ (优化过程中用于简化约束的条件), 这意味着 $P(x)$ 具有因子 x^2 。 $P(x)$ 的最大正根 $r_{\max}$ 即为 $\frac{P(x)}{x^2}$ 的最大正根。由该构造导出的

C_4 的上界为 $r_{\max}^2/(2\pi)$ 。

AlphaEvolve AlphaEvolve 为 $[c_0, c_1, c_2] \approx [0.32925, -0.01159, -6 \times 10^{-5}]$ $P(x) = c_0 H_0(x) + c_1 H_4(x) + c_2 H_8(x)$ 找到
的精化常数为。利用这些系数构造 $P(x)$, 求其最大正根 $r_{\max}$ (通过求 $\frac{P(x)}{x^2}$ 的最大正根并计算 $r_{\max}^2/(2\pi)$ 得到改进的上界 $C_4 < 0.3521$ 。定性地, 我们的线性组合与 [33] 中发现的非常相似, 从而在经验上证实了他们的假设: 该构造几乎是最优的。

注: 在发表本手稿第一版后, Henry Cohn 指出, 在最近的一篇论文 [18] 中, 他们使用了类似但更精细的方法, 得到了更好的常数 0.3284。通过将他们的精细方法融入 AlphaEvolve, 我们将报告的常数进一步改进为 0.3216。详情请参阅随附的 Google Colab。

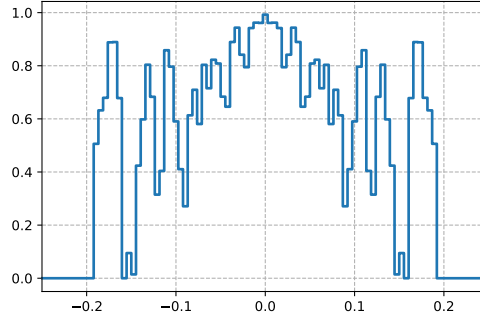


图 10 | AlphaEvolve 为 Erdős 最小重叠问题找到的构造。

B.5. Erdős 最小重叠问题 设 C_5 为满足以下条件的最大常数：

$$\sup_{x \in [-2, 2]} \int_{-1}^1 f(t)g(x+t) dt \geq C_5$$

对所有非负 $f, g: [-1, 1] \rightarrow [0, 1]$ ，其中在 $[-1, 1]$ 上 $f+g=1$ 且 $\int f=1$ ，我们将 f, g 在 $[-1, 1]$ 外延拓为零。该常数控制 [25] 最小重叠问题的渐近性。界

$$0.379005 \leq C_5 \leq 0.380927$$

已知，其中下界是通过凸规划方法在 [107] 中获得的。

已知（见 [40]）该常数等于所有阶跃函数 h 在 $[0, 2]$ 上取值于 $[0, 1]$ 且满足条件下的下确界

$$\max_k \int h(x)(1-h(x+k))dx.$$

随后利用该结果得到了 Erdős 最小重叠问题的上界，在 [40] 中通过阶跃函数构造实现。图 10 所示的阶跃函数比之前的界略好，给出上界 $C_5 \leq 0.380924$ 。

B.6. 有限集的和与差

设 C_6 为满足以下陈述的最大常数：存在任意的整数有限集 A, B ，使得 $|A+B| \ll |A|$ 且 $|A-B| \gg |A+B|^{C_6}$ 。（此处 $A+B = \{a+b : a \in A, b \in B\}$ 和 $A-B = \{a-b : a \in A, b \in B\}$ 分别表示和集与差集。记号 $X \ll Y$ 表示对于某个与集合 A, B 无关的常数 C （对于足够大的集合 A, B ），有 $X < CY$ 。记号 $X \gg Y$ 表示对于某个与集合 A, B 无关的正常数 C' （对于足够大的集合 A, B ），有 $X \geq C'Y$ ）。

$$1.14465 \leq C_6 \leq \frac{4}{3}; \quad (2)$$

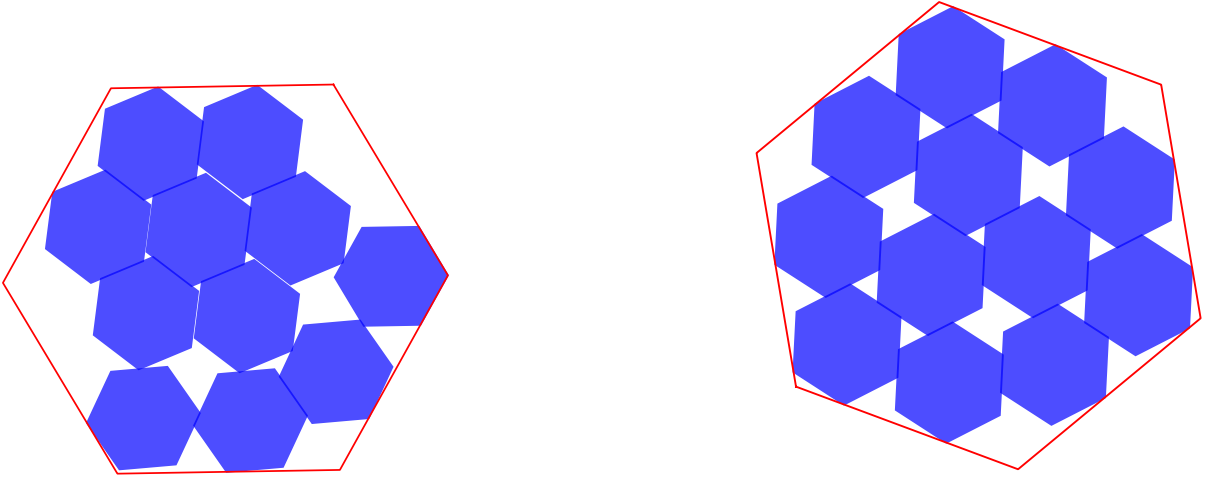


图 11 | AlphaEvolve 发现的填充问题构造。左：将 11 个单位正六边形填入边长为 3.931 的正六边形。右：将 12 个单位正六边形填入边长为 3.942 的正六边形。

上界见 [39, 推论 3], 下界见 [39, 定理 1]。下界的主要工具是 Gyarmati 等人的以下结果 [39]:

$$C_6 \geq 1 + \frac{\log \frac{|U-U|}{|U+U|}}{\log(2 \max(U) + 1)} \quad (3)$$

对于任意包含零且满足 $|U-U| \leq 2 \max(U) + 1$ 的非负整数有限集 U , AlphaEvolve 找到了一个大小为 2003 的集合 U_1 , 将下界提高到 $1.1479 \leq C_6$, 以及另一个大小为 54265 的集合 U_2 , 进一步将下界提高到 $1.1584 \leq C_6$ 。

B.7. 在正六边形内填充单位正六边形

考虑将 n 个边长为单位长度的互不相交的正六边形装入一个更大的正六边形中, 并最小化外六边形的边长。对于 $n = 11$ 和 $n = 12$, 已知的最佳构造分别使用边长为 3.943 和 4.0 的外六边形 [29]。AlphaEvolve 找到了将这两个界分别改进为 3.931 和 3.942 的装填方案。这些方案如图 11 所示。

B.8. 最小化最大与最小距离之比 对于任意 n 和 d , 本问题的目标是在 d 维空间中寻找 n 个点, 以最小化最大与最小成对距离之比。AlphaEvolve 发现了两种新构造, 改进了已知最优界。所发现的构造如图 12 所示。在二维空间中, AlphaEvolve 找到了 16 个点, 其比值为 $\approx$

已知的最佳界限 $\sqrt{\frac{1}{2}}$ 。 (在该参考文献中, 并非使用比值本身, 而是报告比值的平方, 并采用相同的约定。) $\sqrt{12.889266112}$, 提升

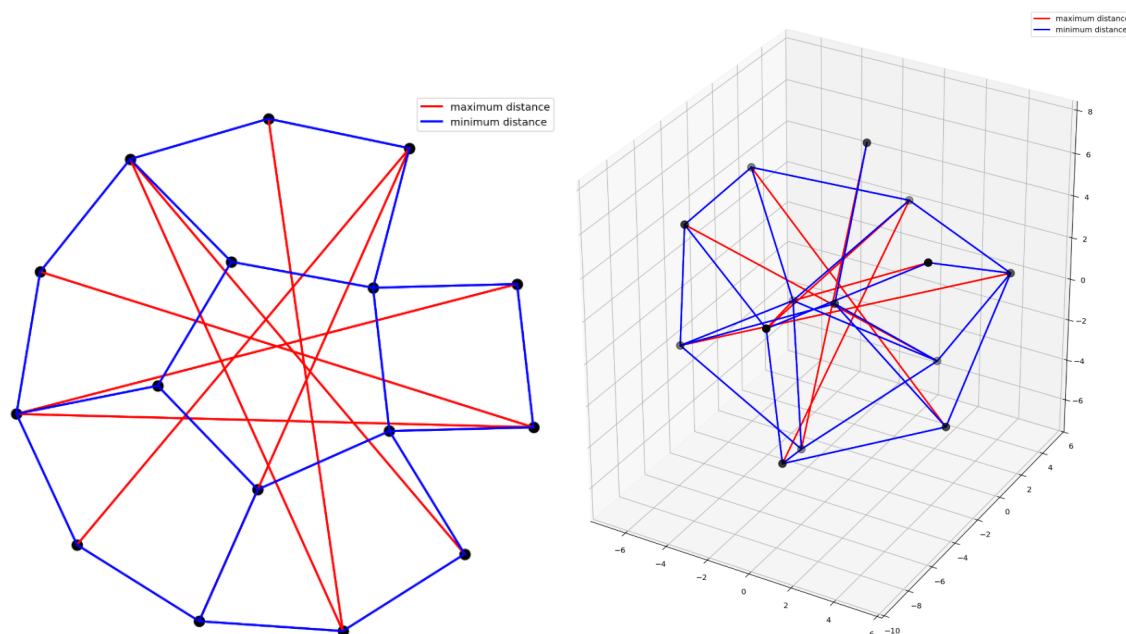


图 12 | 左：二维空间中 16 个点，其最大距离与最小距离之比达到 $\approx \sqrt{12.889266112}$ 。右：三维空间中 14 个点，其比值达到 $\approx \sqrt{4.165849767}$ 。两种构造均改进了已知最优界。

在三维空间中，AlphaEvolve 找到了 14 个 $\sqrt{4.165849767}$ ，改进了已知最佳界限点，其比率为 $\approx \sqrt{4.168}$ [29]。

B.9 三角形的海尔布隆问题

该问题的目标是在单位面积的三角形上或内部找到 n 个点，使得由这些点形成的最小三角形面积最大化。对于 $n = 11$ ，最先进技术（SOTA）为 0.036 [29]，AlphaEvolve 找到了一个最小面积大于 0.0365，的构造，如图 13（左）所示。

B.10 凸区域的海尔布隆问题 该问题的目标是在单位面积的凸区域上或内部找到 n 个点，使得由这些点形成的最小三角形面积最大化。AlphaEvolve 改进了两个已知最佳界限。

对于 $n = 13$ ，最先进技术（SOTA）为 0.0306 [29]，AlphaEvolve 将其改进为 0.0309（见图 13（中））。对于 $n = 14$ ，最先进技术（SOTA）为 0.0277 [29]，AlphaEvolve 将其改进为 0.0278（见图 13（右））。

B.11 11 维接吻数

接吻问题询问有多少个互不相交的单位球可以相切地包裹在给定单位球周围。在 d 维中这样的最大数目称为 d 维接吻数 [8]。对于 $d = 11$ ，已知最佳下界为 592 [31]，AlphaEvolve 将其改进

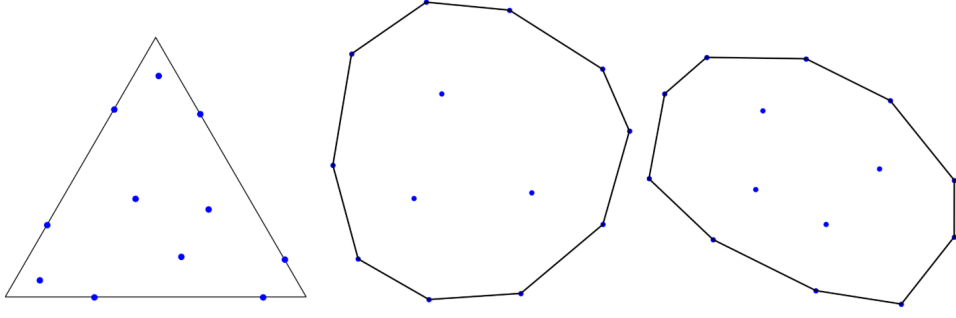


图 13 | AlphaEvolve 发现的新构造，改进了 Heilbronn 问题两个变体的已知最优界。左：单位面积三角形内 11 个点，所有形成的三角形面积 > 0.0365 。中：单位面积凸区域内 13 个点，所有形成的三角形面积 ≥ 0.0309 。右：单位凸区域内 14 个点，最小面积 ≥ 0.0278 。

将此提高到 593。为证明 11 维接吻数的下界为 593，AlphaEvolve 找到了 593 个 11 维非零整坐标点，使得这些点的最大范数小于它们之间的最小成对距离。根据以下引理，这意味着 11 维接吻数至少为 593。

引理 1. 设 $C \subset \mathbb{R}^d$ 为满足 $0 \notin C$ 且

$$\min \{ \|x - y\| : x \neq y \in C \} \geq \max \{ \|x\| : x \in C \}.$$

则以 n 个 $\left\{ \frac{2x}{\|x\|} : x \in C \right\}$ 为球心的单位球构成维度 d 中的有效接吻构型。特别地，维度 d 中的接吻数至少为 $|C|$ 。

证明. 对于任意 $x \neq y \in C$ ，不等式 $\|x - y\|^2 \geq \max \{ \|x\|^2, \|y\|^2 \}$ 蕴含

$$2\langle x, y \rangle \leq \|x\|^2 + \|y\|^2 - \max \{ \|x\|^2, \|y\|^2 \} = \min \{ \|x\|^2, \|y\|^2 \} \leq \|x\| \cdot \|y\|, \quad (4)$$

where the last inequality holds because the minimum of two positive numbers is less than or equal to their geometric mean. The points $\left\{ \frac{2x}{\|x\|} : x \in C \right\}$ have norm 2, so unit spheres centred at them are tangent to a unit sphere centred at the origin. The last step is to show that these spheres do not overlap. This is equivalent to showing, for all $x \neq y \in C$, that

$$\left\| \frac{2x}{\|x\|} - \frac{2y}{\|y\|} \right\| \geq 2.$$

化简后，这等价于 $2\langle x, y \rangle \leq \|x\| \cdot \|y\|$ ，这已在 (4) 中证明。因此

unit spheres centred at $\left\{ \frac{2x}{\|x\|} : x \in C \right\}$ form a valid kissing configuration in dimension d , as required. $\square$

B.12. 在单位正方形内填充圆以最大化半径和 给定正整数 n ，问题是在单位正方形内填充 n 个互不相交的圆，以最大化其半径之和。AlphaEvolve 找到了两个新构造，改进了最新技术 [29]。

对于 $n = 26$ ，最新技术为 2.634，AlphaEvolve 将其改进为 2.635；见图 14（左）。对于 $n = 32$ ，最新技术为 2.936，AlphaEvolve 将其改进为 2.937；见图 14（中）。

B.13. 在周长为 4 的矩形内填充圆以最大化半径之和

给定一个正整数 n ，问题是在周长为 4 的矩形内填充 n 个互不相交的圆，以最大化其半径之和。AlphaEvolve 为 $n = 21$ 找到了一种新的构造，将最新技术从 2.364 [29] 改进到 2.3658；见图 14（右）。

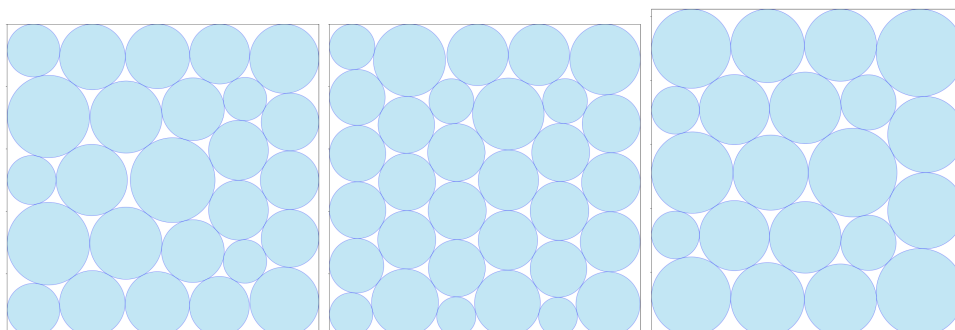


图 14 | AlphaEvolve 发现的新构造，改进了填充圆以最大化半径之和的已知最佳界限。左：单位正方形内 26 个圆，半径之和为 > 2.635 。中：单位正方形内 32 个圆，半径之和为 ≥ 2.937 。右：周长为 4 的矩形内 21 个圆，半径之和为 ≥ 2.365 。